GOOGLE | 1POINT3ACRES | TOPIC 1 OF 9

Arrays, Hashing and Two Pointers

19 questions to study here, each on its own page, with the full interview answer and code

10 more fit this topic but are covered in another topic: they are listed as SKIP at the end of the contents

18	1
MEDIUM	EASY

How to use

- Questions are Hard first, then Medium, then Easy, newest last-asked date first.
- A question that fits several topics has its pages in only one PDF; the other topics list it as SKIP and name where it is.
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Arrays, Hashing and Two Pointers | 19 questions | tap a title to jump to it

M 1. Visible People in a Line (Last Person, Then Everyone) **2026-10-07**
M 2. Longest Non-Decreasing Subarray with One Replacement **2026-10-04**
M 3. Meeting Rooms / Interval Scheduling Variants **2026-09-29**
M 4. Range Duplicate Check with Constant-Time Queries **2026-09-29**
M 5. Active Communication Top Users **2026-09-22**
M 6. Longest Same-Color Grid Sequence **2026-09-08**
M 7. Union of Sorted Interval Lists **2026-08-16**
M 8. Cutting Ribbons (LC 1891) **2026-08-16**
M 9. Target Expressions from Ordered Digits **2026-08-13**
M 10. Progressive Array Partitioning **2026-08-11**
M 11. Piano Hand Movement Grouping **2026-07-14**
M 12. Website Activity Analytics: Unique Users + Session Time **2026-06-26**
M 13. Gold Chain Split into Two Equal Halves **2026-04-21**
M 14. Sum of Good Subarrays (Adjacent Diff ±1) **2026-02-08**
M 15. Two-Day Log Intersection under Memory Limit **2025-11-27**
M 16. Rotate Any m×n 2D Matrix
M 17. Jump Game with Profit
M 18. Russian Doll Envelopes
E 19. Robot Status Message Deduplication Window **2026-09-10**

SKIP: these fit this topic but have their pages in another topic (10)

H SKIP Maximum-Area Rectangle from Planar Points **topic 8: Greedy and Math**
H SKIP Necklace Cut into Two Halves with Equal D/R Counts **topic 2: Strings, Parsing and Recursion**
M SKIP Connected Crop Arrangement on a Grid **topic 6: Grids and Simulation**
M SKIP Temperature Monitoring Data Structure **topic 9: Design, Streaming and Scale**
M SKIP Next-Word Predictor **topic 2: Strings, Parsing and Recursion**
M SKIP Top-K Frequent in Large Logs **topic 7: Heaps, Stacks, Sorting and Binary Search**
M SKIP Streaming Points: Emit K Within Distance **topic 9: Design, Streaming and Scale**
M SKIP Greedy Longest-Match Dictionary Tokenizer **topic 2: Strings, Parsing and Recursion**
M SKIP Employee Hierarchy Add / Delete **topic 5: Trees and Tries**
M SKIP Run-Length Vector Storage + Dot Product **topic 9: Design, Streaming and Scale**

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Heaps, Stacks, Sorting and Binary Search (listed there as SKIP, no pages). Site tags: array, monotonic-stack, stack, medium

MEDIUM | 1Point3Acres | tech-screen | 1 of 19

Visible People in a Line (Last Person, Then Everyone)

ASKED 2026-10-07 | 13 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 181/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1944 Number of Visible People in a Queue

LeetCode match: LC 1944 Number of Visible People in a Queue (related)

Frequency: Single report | Last asked: 2026-10-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/visible-people-in-line

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

Input: an array of heights representing a line of people.
Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person,
    height 5, can see 3, 4, 2, 8, 9, 10).
```

APPROACH

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what blocks the view (someone strictly taller than both), how ties are treated, which directions count in Part 2, and whether the code runs (it does not: trace by hand).

BRUTE FORCE TO SAY OUT LOUD

Check every pair against the people between them. O(n^2) for the last person, O(n^3) for everyone.

OPTIMAL IDEA

Part 1: walk from the person before the last towards the front, keeping the maximum height between the last person and the current one; the current person is visible when that maximum is at most the larger of the two heights. O(n). Part 2: for each person, everyone up to the first strictly taller person on the right is visible, then only people at least as tall as everyone from there on; a monotonic stack gives the next at-least-as-tall and next strictly-taller indexes and the chain length in O(n). Run it on the reversed line for the left side and add.

DRY-RUN THESE CASES

A single person, all equal heights, strictly increasing and decreasing lines, the example from the statement (the last person sees 6), a tie case you invent.

LIKELY FOLLOW-UPS

Only looking in one direction, blocking on equal heights, or returning who they see instead of how many.

PITFALLS

Using a strict comparison where the rule says taller than both. Forgetting that the first taller person is itself visible. Rereading the example without tracing it by hand.

HOW TO THINK ABOUT IT

Person A sees person B when nobody strictly between them is taller than both, which means the tallest person between them is at most as tall as the taller of A and B. Walking away from the last person and keeping the running maximum of everyone passed lets you test each person in O(1). For everyone at once the visible people to the right form a chain: everything up to the first taller person, then only the people who are at least as tall as everyone before them. A monotonic stack builds those chains for all positions in O(n).

Basis: LeetCode 1944, Number of Visible People in a Queue, with the rule that a person is hidden only by someone taller than both.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def sees(a, i, j):
    lo, hi = min(i, j), max(i, j)
    for k in range(lo + 1, hi):
        if a[k] > max(a[i], a[j]):
            return False
    return True
def count_for(a, i):
    return sum(1 for j in range(len(a))
               if j != i and sees(a, i, j))
```

TIME	SPACE
O(n^3) for everyone, O(n^2) for the last person	**O(1)**

BRUTE FORCE

WHY IT IS SLOW

Each pair rescans the people between it, O(n^3) for everyone and O(n^2) for the last person. One pass with a running maximum answers the last person in O(n), and a monotonic stack answers everyone in O(n).

OPTIMAL SOLUTION · One pass with a running maximum (last person)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 2: return the number of people each person can see.

Run the right-looking count for every position with a monotonic stack, run it again on the reversed line for the looking-left count, and add the two. The stack makes both passes O(n), instead of the O(n^2) you would get by repeating the last-person scan for every position.

FOLLOW-UP 2 How do ties and the direction of sight change the answer?

Under the stated rule only someone strictly taller than both blocks the view, so equal heights never block: in [5, 5, 5] the first person sees both others. If the interviewer wants 'taller or equal blocks', replace the non-strict comparisons with strict ones. If a person only looks towards the front, drop the reversed pass and use see_right on the reversed line alone.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
INF = float("inf")


# BASE QUESTION (optimal): One pass with a running maximum
# (last person)
def last_sees(a):
    """Part 1: walk away from the last person keeping the
    tallest height seen strictly between us and them."""
    if not a:
        return 0
    last, between, count = a[-1], -INF, 0
    for k in range(len(a) - 2, -1, -1):
        # nobody between is taller than both of us
        if between <= max(last, a[k]):
            count += 1
        between = max(between, a[k])
    return count


# FOLLOW-UP 1: Part 2: return the number of people each
# person can see.
def see_right(a):
    """Part 2, looking right: everyone up to the first
    strictly taller person is visible, then only people at
    least as tall as everyone from that person on."""
    n = len(a)
    next_ge, next_gt = [None] * n, [None] * n
    stack = []
    for k in range(n - 1, -1, -1):
        # next person at least as tall as person k
        while stack and a[stack[-1]] < a[k]:
            stack.pop()
        next_ge[k] = stack[-1] if stack else None
        stack.append(k)
    stack = []
    for k in range(n - 1, -1, -1):
        # next person strictly taller than person k
        while stack and a[stack[-1]] <= a[k]:
            stack.pop()
        next_gt[k] = stack[-1] if stack else None
        stack.append(k)
    chain = [0] * n
    for k in range(n - 1, -1, -1):
        # how many "at least as tall" steps follow k
        if next_ge[k] is not None:
            chain[k] = 1 + chain[next_ge[k]]
    out = []
    for i in range(n):
        t = next_gt[i]
        if t is None:
            out.append(n - 1 - i)
        else:
            out.append(t - i + chain[t])
    return out


# FOLLOW-UP 1: Part 2: return the number of people each
# person can see.
def see_all(a):
    # looking left is looking right on the reversed line
    left = see_right(a[::-1])[::-1]
    return [x + y for x, y in zip(left, see_right(a))]


# CHECKER (brute force)
def sees(a, i, j):
    # the oracle: nobody strictly between is taller than
    # both
    lo, hi = min(i, j), max(i, j)
    return all(a[k] <= max(a[i], a[j])
               for k in range(lo + 1, hi))


# CHECKER (brute force)
def counts_brute(a):
    return [sum(1 for j in range(len(a))
                if j != i and sees(a, i, j))
            for i in range(len(a))]


if __name__ == "__main__":
    line = [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
    assert last_sees(line) == 6
    assert see_all(line)[-1] == 6
    assert last_sees([]) == 0 and last_sees([7]) == 0
    # equal heights never block each other
    assert last_sees([5, 5, 5]) == 2
    assert see_all([5, 5, 5]) == [2, 2, 2]
    import random
    for _ in range(300):
```

```
        a = [random.randint(1, 6)
             for _ in range(random.randint(0, 9))]
        assert see_all(a) == counts_brute(a)
        if a:
            assert last_sees(a) == counts_brute(a)[-1]
```

NOTES FROM CANDIDATES

Pitfalls: The code is written in a plain text editor that does not compile or run; trace the example by hand before declaring the solution done.
Confirm the direction of "see" for Part 2 (toward the front only, or both directions) and how equal heights are treated before coding; the example does not exercise ties.

Follow-ups: Generalize from the last person to every person in the line (Part 2).

PREPARATION

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Dynamic Programming (listed there as SKIP, no pages). Site tags: array, dp, two-pointer, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 2 of 19

Longest Non-Decreasing Subarray with One Replacement

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 209/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-non-decreasing-subarray-one-replacement

QUESTION

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Some rounds start from the plain longest-increasing-subarray question and add the one-replacement follow-up; those versions ask for an increasing run, so clarify strict vs non-strict.

Input: an integer array nums.
You may change at most one element to any value.
Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

Variant: other rounds ask for a strictly increasing run and open with the no-replacement version (longest increasing subarray) before adding the one-replacement follow-up — clarify strict vs non-strict before coding.

Expect to discuss and compare the complexity of several approaches; a brute-force first pass is usually followed by a request to optimize.

EXAMPLES

```
nums = [1, 0, 3, 4, 5, 2, 3]
Output: 5
Change nums[1] from 0 to 1 -> [1, 1, 3, 4, 5, 2, 3]
[1, 1, 3, 4, 5] is a non-decreasing subarray of
    length 5.
```

APPROACH

Write the plain longest-run scan, then extend it with left-run / right-run arrays (or a two-state DP: "no change used" / "change used") so you can explain the jump from one to the other.
Test the bridge case explicitly: joining the run ending at i - 1 with the run starting at i + 1 is only valid when nums[i - 1] <= nums[i + 1]; in the strict variant there must be room for a value strictly between the neighbors, i.e. nums[i + 1] - nums[i - 1] >= 2 when the replacement must be an integer (plain < only if any real value is allowed). Otherwise the change only extends one side by one.
Repeat the exercise with the strict-increasing variant to make sure your comparisons flip correctly.

HOW TO THINK ABOUT IT

Precompute left[i], the length of the non-decreasing run ending at i, and right[i], the length of the run starting at i. If you change element i to any value, it can join the run on its left and the run on its right when a[i-1] <= a[i+1] (a value between them exists); otherwise it can join only the longer of the two. Take the best over all i.

Basis: Written for this question (the one-change version of LeetCode 674, Longest Continuous Increasing Subsequence).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    best = longest_non_decreasing(a)
    for i in range(len(a)):
        for v in candidate_values(a):
            b = a[:i] + [v] + a[i + 1:]
            run = longest_non_decreasing(b)
            best = max(best, run)
    return best
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^3). The run lengths on both sides of each position, computed once, answer every change in O(1).

OPTIMAL SOLUTION · Left and right run lengths

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The run must be strictly increasing.

Use < in the run lengths, and when the replaced value must fit between the neighbours check a[i+1] - a[i-1] >= 2 (an integer strictly between them exists) instead of a[i-1] <= a[i+1]. Clarify strict versus non-strict before coding.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
def run_lengths(a, ok):
    n = len(a)
    # left[i]: run ending at i, right[i]: starting at i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if ok(a[i - 1], a[i]):
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ok(a[i], a[i + 1]):
            right[i] = right[i + 1] + 1
    return left, right


# BASE QUESTION (optimal): Left and right run lengths
# FOLLOW-UP 1: The run must be strictly increasing.
def longest_with_one_change(a, strict=False):
    n = len(a)
    # nothing to fix in arrays of length 0, 1 or 2
    if n <= 2:
        return n
    # FOLLOW-UP 1: strict=True means strictly increasing
    # non-decreasing uses <=, strictly increasing uses <
    def ok(x, y):
        return x < y if strict else x <= y

    # can a value fit between the two neighbours?
    gap = 2 if strict else 0
    left, right = run_lengths(a, ok)
    best = 1
    # try changing each element a[i]
    for i in range(n):
        L = left[i - 1] if i else 0
        R = right[i + 1] if i + 1 < n else 0
        # at an end, or when a value fits between the
        # neighbours, the change joins both runs
        if not L or not R or a[i + 1] - a[i - 1] >= gap:
            best = max(best, L + 1 + R)
        # otherwise it joins only the longer run
        else:
            best = max(best, max(L, R) + 1)
    return best


# WARM-UP: no replacement allowed
def longest_without_change(a):
```

```
    # the plain longest increasing subarray warm-up
    best = run = 0
    for i, x in enumerate(a):
        run = run + 1 if i and a[i - 1] < x else 1
        best = max(best, run)
    return best


if __name__ == "__main__":
    # change 0 to 1: [1, 1, 3, 4, 5] has length 5
    nums = [1, 0, 3, 4, 5, 2, 3]
```

```
    assert longest_with_one_change(nums) == 5
    # tiny arrays
    assert longest_with_one_change([]) == 0
    assert longest_with_one_change([7]) == 1
    # strict version needs room between neighbours
    strict = longest_with_one_change
    assert strict([1, 2, 2, 3, 4], True) == 4
    assert strict([1, 1, 1, 1], True) == 2
    # the warm-up without any change
    assert longest_without_change([1, 3, 5, 4, 7]) == 3
```

NOTES FROM CANDIDATES

Asked as an onsite coding round in an early-career L3 loop, and as the first technical round of new-grad VOs.
Candidates who wrote a slow version first were asked to optimize, leaving little time for anything else — the problem is short, so interviewers expect the linear approach to arrive quickly.
Edge cases worth confirming: arrays of length 0–2, where the replaced element is the first or last of the run, and whether the replaced value must fit between both neighbors or only one.

PREPARATION

Write the plain longest-run scan, then extend it with left-run / right-run arrays (or a two-state DP: "no change used" / "change used") so you can explain the jump from one to the other.
Test the bridge case explicitly: joining the run ending at i - 1 with the run starting at i + 1 is only valid when nums[i - 1] <= nums[i + 1]; in the strict variant there must be room for a value strictly between the neighbors, i.e. nums[i + 1] - nums[i - 1] >= 2 when the replacement must be an integer (plain < only if any real value is allowed). Otherwise the change only extends one side by one.
Repeat the exercise with the strict-increasing variant to make sure your comparisons flip correctly.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Heaps, Stacks, Sorting and Binary Search, Greedy and Math (listed there as SKIP, no pages). Site tags: interval, heap, greedy, medium

MEDIUM | IN APP LC 253 | 1Point3Acres | phone-screen, tech-screen, onsite-coding | 3 of 19

Meeting Rooms / Interval Scheduling Variants

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE **ALL TIME 58** **30 DAYS 0** **PRIORITY 44/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: your app's interview approach (LeetCode #253)

LeetCode match: LC 253 Meeting Rooms II

Frequency: Very high | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

QUESTION

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

QUESTION

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling). Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example:
    [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40],
    D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap:
    intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

APPROACH

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

HOW TO THINK ABOUT IT

The number of rooms needed is the largest number of intervals overlapping at any moment. Turn each interval into +1 at its start and -1 at its end, sort the events (ends before starts at the same time) and keep a running count. To say which room each meeting gets, keep a min-heap of (end time, room) and reuse the room that frees up first.

Basis: LeetCode 253, Meeting Rooms II, with the common variants: room assignment and cyclic (24-hour) time.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(intervals):
    best = 0
    for start, _ in intervals:
        busy = 0
        for s, e in intervals:
            if s <= start < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(n^2)	O(1)

BRUTE FORCE

WHY IT IS SLOW

Every interval is compared with every other, O(n^2). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Sweep line over start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Meetings wrap around midnight (cyclic time).

Split every wrapped meeting [s, e) with s >= e into [s, 24h) and [0, e) and run the same sweep: the overlap at any clock time is unchanged. A meeting that is exactly 24 hours long covers the whole day.

FOLLOW-UP 2 The input is huge and spread over many machines.

Each shard sorts its events and produces a compressed timeline of (time, delta) changes; merge the sorted timelines with a k-way merge and take the running maximum, or add the per-shard counts at fixed time buckets. The answer is a max over time of a sum over shards.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from collections import Counter


# BASE QUESTION (optimal): Sweep line over start and end
# events
def min_rooms(intervals):
    # +1 at every start, -1 at every end
    events = []
    for s, e in intervals:
        events.append((s, 1))
        events.append((e, -1))
    # at equal times ends (-1) sort before starts (+1)
    events.sort()
    cur = best = 0
    for _, delta in events:
        cur += delta
        best = max(best, cur)
    return best


# VARIANT: which room for each meeting
def assign_rooms(intervals):
    # which room does each meeting get?
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
```

```
    heap, rooms, used = [], [0] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        # reuse the room that frees first, if it is free
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# FOLLOW-UP 1: Meetings wrap around midnight (cyclic
# time).
def min_rooms_cyclic(intervals, day=24 * 60):
    parts = []
    for s, e in intervals:
        if s < e:
```

```
            parts.append((s, e))
        else:
            # wraps past midnight: split at the boundary
            parts += [(s, day), (0, e)]
    return min_rooms(parts)


# VARIANT: on-call consolidation
def oncall_segments(rotations):
    # rotations: (name, start, end), half-open intervals
    delta = {}
    for name, s, e in rotations:
        delta.setdefault(s, []).append((name, 1))
        delta.setdefault(e, []).append((name, -1))
    active, out, times = Counter(), [], sorted(delta)
    for t, nxt in zip(times, times[1:]):
        # apply every change that happens at time t
        for name, d in delta[t]:
            active[name] += d
        who = frozenset(n for n in active if active[n])
        # skip gaps; merge with the previous equal segment
        if not who:
            continue
        if out and out[-1][1] == t and out[-1][2] == who:
            out[-1] = (out[-1][0], nxt, who)
        else:
            out.append((t, nxt, who))
    return out


if __name__ == "__main__":
    base = [(0, 30), (5, 10), (15, 20)]
    assert min_rooms(base) == 2
    # touching meetings share a room
    assert min_rooms([(5, 8), (8, 9)]) == 1
    rooms = assign_rooms(base)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    wrap = [(22 * 60, 2 * 60), (23 * 60, 60), (0, 3 * 60)]
    assert min_rooms_cyclic(wrap) == 3
    rot = [("A", 1, 5), ("B", 3, 8)]
    assert oncall_segments(rot) == [
        (1, 3, frozenset("A")),
        (3, 5, frozenset("AB")),
        (5, 8, frozenset("B"))]
    # a gap with nobody on call is left out
```

```
    gap = [("A", 1, 2), ("B", 4, 5)]
    assert len(oncall_segments(gap)) == 2
```

NOTES FROM CANDIDATES

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.

The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

PREPARATION

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Site tags: array, sliding-window, hashmap, prefix-sum, medium

MEDIUM | 1Point3Acres | onsite-coding | 4 of 19

Range Duplicate Check with Constant-Time Queries

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 211/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/range-duplicate-check-constant-time-queries

QUESTION

Decide whether the elements in a range of an array are all unique. Progression: set-based duplicate check -> sliding window -> many range queries where reads dominate, requiring O(1) per query after linear-time preprocessing / updates with linear space.

Base: given an array and a range [l, r], determine whether the elements in that range are all unique.
The problem is staged:

Easy: duplicate detection with a set.
Sliding-window duplicate checking.
Many queries over different ranges: optimize for a read-heavy, write-light workload.

Target for the final stage: O(1) per query, preprocessing (and updates, in one version) in linear time, linear space.

APPROACH

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

HOW TO THINK ABOUT IT

A range [l, r] has a duplicate exactly when some index i in it has its next equal element at an index <= r. Let next[i] be the next index holding the same value and reach[l] the smallest next[i] over all i >= l. Then the range is all-unique iff reach[l] > r, a single comparison per query after one linear pass.

Basis: Written for this question (a precomputed next-occurrence array, similar in spirit to LeetCode 3, Longest Substring Without Repeating Characters).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def all_unique(a, l, r):
    seen = set()
    for x in a[l:r + 1]:
        if x in seen:
            return False
        seen.add(x)
    return True
```

TIME	SPACE
O(r - l) per query	O(r - l)

WHY IT IS SLOW

Each query costs the length of the range. A next-occurrence array with suffix minimums answers each query with one comparison.

OPTIMAL SOLUTION · Next-occurrence array with suffix minimum

TIME	SPACE
O(n) preprocessing, O(1) per query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array changes between queries.

Rebuild next and reach in O(n) after a batch of changes when reads dominate. If single-element updates are frequent, keep per-value sorted index lists and a segment tree of next-occurrence minima so an update and a query cost O(log n) instead.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import Counter

INF = float("inf")


# EASY VERSION (say it first)
def unique_with_set(a, l, r):
    # easy version: put the range in a set
    return len(set(a[l:r + 1])) == r - l + 1


# STAGE 2: sliding window
def longest_unique_window(a):
    # stage 2: sliding window with a counter
    seen, left, best = Counter(), 0, 0
    for right, x in enumerate(a):
```

```
        seen[x] += 1
        # shrink until x occurs only once
        while seen[x] > 1:
            seen[a[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best


# BASE QUESTION (optimal): Next-occurrence array with
# suffix minimum
# FOLLOW-UP 1: The array changes between queries.
class RangeUnique:
    """O(n) build, then O(1) per query."""

    def __init__(self, a):
        self.a = list(a)
        self.rebuild()

    def rebuild(self):
        a, n = self.a, len(self.a)
        # nxt[i] = next index holding the same value
        nxt, last = [INF] * n, {}
        for i in range(n - 1, -1, -1):
            nxt[i] = last.get(a[i], INF)
            last[a[i]] = i
        # reach[i] = earliest repeat from index i onward
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
            self.reach[i] = min(nxt[i], self.reach[i + 1])

    def unique(self, l, r):
        # no repeat starts in [l, r] and ends inside it
        return self.reach[l] > r

    # FOLLOW-UP 1: the array changes between queries
    def update(self, i, value):
        # reads dominate, so a linear rebuild is fine
        self.a[i] = value
        self.rebuild()


if __name__ == "__main__":
    a = [1, 2, 3, 1, 4]
    q = RangeUnique(a)
    for l in range(len(a)):
        for r in range(l, len(a)):
            plain = unique_with_set(a, l, r)
            assert q.unique(l, r) == plain
    assert q.unique(0, 2) and not q.unique(0, 3)
    assert longest_unique_window(a) == 4
    # an update can create or remove a duplicate
    q.update(3, 9)
    assert q.unique(0, 4)
    q.update(0, 9)
    assert not q.unique(0, 3)
```

NOTES FROM CANDIDATES

In one L4 onsite, this was the second question of a round that opened with weighted job scheduling; the interviewer asked specifically for the precomputation that makes each query O(1). The interviewer states the complexity targets explicitly, so the round is judged on hitting them, not on a working brute force. Clarify whether the array can change between queries (one version mentions linear-time updates) and whether ranges are inclusive.

PREPARATION

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Heaps, Stacks, Sorting and Binary Search, Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: hashmap, set, sorting, top-k, streaming, medium

MEDIUM | 1Point3Acres | phone-screen | 5 of 19

Active Communication Top Users

ASKED 2026-09-22 | 28 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 228/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-22 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/active-communication-top-users

QUESTION

Process a continuous stream of user pairs such as **`A-B`** and **`B-C`**; each pair creates an active communication edge, and the system should output users ordered by current communication count. Follow-ups add self-communication handling, duplicate communication dedupe, and top-K users by count. An additional API variant exposes register and mostActiveUser(), then asks for constant-time most-active lookup and a changing-K ranking extension.SWEhashmapsetsortingtop-kstreamingmediumFrequencyLowLast asked2026-09-22Stagephone-screen

APPROACH

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a pair A-B counts for both users, whether repeated pairs count once, how self communication is treated, how ties order, and whether k changes over time.

BRUTE FORCE TO SAY OUT LOUD

Recount from all edges and sort users on every query. O(E log U).

OPTIMAL IDEA

Normalize pairs (sorted tuple). Keep an edge set for deduplication, a dict of user to count, and either a sorted structure of (count, user) or a bucket map count to ordered users so an update moves a user by one bucket. Top-K reads the highest buckets. Note the edge set can use quadratic space in the number of users; say so.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

A-B then B-A (duplicate), a self pair, ties on count, a new user, k larger than the number of users.

LIKELY FOLLOW-UPS

Constant-time most active user (track the maximum count with buckets), a changing k, or expiry of old edges.

PITFALLS

Claiming linear space while storing all pairs. Counting a duplicate edge twice.

HOW TO THINK ABOUT IT

Normalize the pair first (sorted tuple) so A-B and B-A are one edge, then keep an edge set for dedup and a count per user. The ranking part is a separate question: top-K comes from a heap, and the single most active user can be maintained in O(1) because counts only ever go up. Mention that the edge set uses quadratic space in the worst case.

Basis: Written. Assumed rules (confirm them): a pair 'A-B' is an unordered communication edge; repeated pairs count once; a user's activity is the number of distinct partners; order by activity descending, then name; self pairs (A-A) are ignored unless a flag says otherwise.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
pairs = set()
def add(a, b):
    pairs.add((min(a, b), max(a, b)))
def top_users():
    partners = defaultdict(set)
    for a, b in pairs:
        partners[a].add(b)
        partners[b].add(a)
    def rank(u):
        return (-len(partners[u]), u)
    return sorted(partners, key=rank)
```

TIME	SPACE
O(E + U log U) per query	**O(E)**

WHY IT IS SLOW

Each query rebuilds all counts and sorts, O(E + U log U). Maintaining counts incrementally with count buckets gives O(1) updates and an O(1) most-active lookup.

OPTIMAL SOLUTION · Edge set, counts per user, count buckets for the maximum

TIME	SPACE
O(1) per pair, O(U log k) for top-K, O(1) for the most active user	**O(E)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Self communication (A-A) should now count.

Pass a flag to the class: a self pair is one edge that adds one to that user's count (and only once). The dedup set already handles repeated self pairs.

FOLLOW-UP 2 Top-K with a changing K, or a fixed-size priority queue.

A heap of size K can be wrong after counts change, because an element outside it can overtake one inside. Re-run heapq.nsmallest on the count dict per query (O(U log K)) or maintain a sorted structure keyed by (count, name) updated on every increment.

FOLLOW-UP 3 The space claim.

Distinct pairs can number U^2 / 2 in the worst case, so the edge set is quadratic in the users; the counts alone are linear. If duplicates are not a concern, drop the set and keep O(U).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from collections import Counter, defaultdict


# BASE QUESTION (optimal): Edge set, counts per user,
# count buckets for the maximum
# FOLLOW-UP 1: Self communication (A-A) should now count.
class Active:
    def __init__(self, count_self=False):
        # distinct edges seen so far (for the dedupe rule)
        self.edges = set()
        # user -> how many different partners
        self.count = Counter()
        # count -> users having exactly that count
        self.bucket = defaultdict(set)
        # the highest count right now (it never drops)
        self.top = 0
        self.count_self = count_self

    def bump(self, user):
        c = self.count[user]
        # move the user from bucket c to bucket c + 1
        if c:
            self.bucket[c].discard(user)
        self.count[user] = c + 1
        self.bucket[c + 1].add(user)
        self.top = max(self.top, c + 1)

    def add(self, a, b):
        # A-B and B-A are the same edge
        pair = (min(a, b), max(a, b))
        # follow-ups: skip self pairs (unless counted)
        # and skip a repeated pair
        skip_self = a == b and not self.count_self
        if skip_self or pair in self.edges:
            return
        self.edges.add(pair)
        self.bump(a)
        # a self pair counts once, not twice
        if a != b:
            self.bump(b)

    def most_active(self):
        # O(1): read the top bucket
        if not self.top:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
        # k users by count, ties broken by name
        def rank(u):
            return (-self.count[u], u)
        return heapq.nsmallest(k, self.count, key=rank)


if __name__ == "__main__":
    a = Active()
    for x, y in [("A", "B"), ("B", "C"), ("B", "A")]:
        a.add(x, y)
    # B talks to A and C; the repeat B-A is ignored
    assert a.most_active() == {"B"}
    assert a.count["B"] == 2 and a.count["A"] == 1
    assert a.top_k(2) == ["B", "A"]
    # self pairs are ignored by default
    a.add("C", "C")
    assert a.count["C"] == 1
    # ... but can be switched on, counting once
    s = Active(count_self=True)
    s.add("C", "C")
    s.add("C", "C")
    assert s.count["C"] == 1
    assert Active().most_active() == set()
```

NOTES FROM CANDIDATES

A Java solution used an ordered map / tree structure to keep counts sortable.
The interviewer asked many clarification questions and expected dry runs.
The first version did not require self-communication handling; it was added as a coded follow-up.
Account for the per-user contact sets when explaining space use: the distinct relationship state can take quadratic space in the number of users. The interviewer explicitly challenged an initial linear-space claim.
A fixed-size priority-queue proposal did not finish the changing-K discussion; the interviewer asked for further optimization.
Maintaining the API variant
With registrations only and any tied maximum acceptable, normalize undirected pairs and store the distinct edges. For a new edge, increment each distinct endpoint once; an accepted self-edge changes one count, while an ignored self-edge changes none. Clarify that choice and the empty-state response.
Cache a user with the largest count. Counts never decrease, so only the endpoints of a new edge can displace the cached winner; duplicate edges do not change it. Reading this cache makes mostActiveUser() O(1), with expected O(1) registration work using

NOTES FROM CANDIDATES

hash-based edge membership and counters under unit-cost user identifiers. A stricter tie rule requires separate handling.
For changing K, one option is a balanced ordered index of all users keyed by count and an agreed tie-breaker. Remove and reinsert the affected users when counts change. Updates cost O(log U); starting at the highest-ranked user and walking entries in descending-count order costs O(log U + K) for at most U returned users. This supports different K values without rebuilding a fixed-capacity heap and is a preparation option, not a specified interviewer solution.
Counters, the maximum cache, and the ranking index use O(U) space. Distinct relationship storage adds O(E), giving O(U + E) overall and O(U^2) in a dense graph.

PREPARATION

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: array, grid

MEDIUM | 1Point3Acres | phone-screen | 6 of 19

Longest Same-Color Grid Sequence

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 148/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix (related)

Frequency: Single report | Last asked: 2026-09-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

QUESTION

Given a grid of colors, return the longest consecutive same-color sequence along a horizontal, vertical, or diagonal line.

Input is a two-dimensional grid of colors such as R, G, and B. Return the length of the longest consecutive sequence of a single color horizontally, vertically, or diagonally.

EXAMPLES

```
R R R G
R R B B
G R R R
B G B R

Answer: 4
```

APPROACH

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

HOW TO THINK ABOUT IT

A straight line can run in four directions (right, down, down-right, down-left). For each cell and each direction keep the length of the same-colour run that ends there: it is the predecessor's run plus one if the colours match, otherwise 1. Because every run only looks one cell back in a fixed direction, one pass in reading order is enough.

Basis: LeetCode 562, Longest Line of Consecutive One in Matrix, generalised to any colour.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = 0
    for r, c in all_cells(grid):
        for dr, dc in DIRECTIONS:
            n = 1
            while same(grid, r, c,
                       r + n * dr, c + n * dc):
                n += 1
            best = max(best, n)
    return best
```

TIME	SPACE
O(m * n * max(m, n))	O(1)

WHY IT IS SLOW

Re-walks the same cells in every line, O(m * n * max(m, n)). Keeping the run length ending at each cell in each direction is O(m * n).

OPTIMAL SOLUTION · Four run lengths per cell

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The line may also bend (a connected region).

Then it is a different problem: the largest connected same-colour region, found with BFS/DFS or union-find in O(m * n). The run-length DP only handles straight lines because each run depends on exactly one predecessor.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# BASE QUESTION (optimal): Four run lengths per cell
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n, best = len(g[0]), 0
    # run lengths ending in the previous row:
    # straight down, down-left, down-right
    up, dl, dr = [0] * n, [0] * n, [0] * n
    for i, row in enumerate(g):
        nu, nl, nr = [1] * n, [1] * n, [1] * n
        left = 0
        for j, color in enumerate(row):
            # horizontal run from the cell on the left
            same = j and row[j - 1] == color
            left = left + 1 if same else 1
            if i:
                # extend the run from the cell above
                if g[i - 1][j] == color:
                    nu[j] = up[j] + 1
```

```
                # from above-left (down-right diagonal)
                if j and g[i - 1][j - 1] == color:
                    nr[j] = dr[j - 1] + 1
                # from above-right (down-left diagonal)
                if j + 1 < n and g[i - 1][j + 1] == color:
                    nl[j] = dl[j + 1] + 1
            best = max(best, left, nu[j], nl[j], nr[j])
        up, dl, dr = nu, nl, nr
    return best


# FOLLOW-UP 1: The line may also bend (a connected
# region).
def largest_region(g):
    # follow-up: the line may bend (connected region)
    seen, best = set(), 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack, size = [(r, c)], 0
            while stack:
                x, y = stack.pop()
                size += 1
                for a, b in ((x + 1, y), (x - 1, y),
```

```
                             (x, y + 1), (x, y - 1)):
                    if a < 0 or b < 0:
                        continue
                    if a >= len(g) or b >= len(g[0]):
                        continue
                    if (a, b) not in seen \
                            and g[a][b] == g[r][c]:
                        seen.add((a, b))
                        stack.append((a, b))
            best = max(best, size)
    return best


if __name__ == "__main__":
    grid = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_same_color_line(grid) == 4
    assert longest_same_color_line([]) == 0
    # one column and alternating colours
    assert longest_same_color_line(["R", "R", "R"]) == 3
    assert longest_same_color_line(["RGR", "GRG"]) == 2
    # a bent region is longer than any straight line
    bent = ["RRR", "GGR"]
    assert longest_same_color_line(bent) == 3
    assert largest_region(["RR", "GR"]) == 3
```

NOTES FROM CANDIDATES

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

PREPARATION

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.

Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Heaps, Stacks, Sorting and Binary Search, Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: interval, two-pointer, heap, streaming, medium

MEDIUM | 1Point3Acres | onsite-coding | 7 of 19

Union of Sorted Interval Lists

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK **ALL TIME 25.5** **30 DAYS 0** **PRIORITY 139/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #986 Interval List Intersections

LeetCode match: LC 986 Interval List Intersections (related)

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/sorted-interval-list-union

QUESTION

Reverse of LC 986: given two individually sorted, internally non-overlapping interval lists, return their union. Follow-ups extend to K lists with a heap, inputs too large for memory, and lists that may overlap internally.SWEintervaltwo-pointerheapstreamingmediumFrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether intervals are closed or half-open, whether touching intervals merge, whether each list is sorted and non-overlapping internally (later follow-up removes this), and how large the inputs are.

BRUTE FORCE TO SAY OUT LOUD

Concatenate both lists, sort, and merge. O((m + n) log (m + n)).

OPTIMAL IDEA

Two-pointer merge: repeatedly take the interval with the smaller start from either list and fold it into the last output interval if it overlaps (start at most the previous end), else append it. O(m + n). For K lists use heapq.merge (a heap of K heads), and since it is a generator the inputs never need to fit in memory.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

One list empty, no overlaps, one interval swallowing many, touching endpoints, identical lists.

LIKELY FOLLOW-UPS

K lists, lists too big for memory, lists that overlap internally (sort each first or heap everything).

PITFALLS

Using end instead of start to pick the next interval. Not taking the maximum of ends when merging.

HOW TO THINK ABOUT IT

Each list is already sorted, so you can walk both with two pointers like the merge step of merge sort: always take the interval with the smaller start next. Once the intervals come out in start order, merging is the usual one-pass fold: extend the last output interval if the next one overlaps or touches it, otherwise start a new one.

Basis: The reverse of LeetCode 986 (Interval List Intersections): union instead of intersection.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(list1, list2):
    merged = []
    for s, e in sorted(list1 + list2):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME	SPACE
O((m + n) log (m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that both lists are already sorted: O((m + n) log (m + n)) instead of O(m + n) for a two-pointer merge.

OPTIMAL SOLUTION · Two-pointer merge

TIME	SPACE
O(m + n)	O(m + n) for the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the lists are too big for memory?

Read each list through an iterator (a file or a paged API). union_k only ever holds one interval per list plus the current merged interval, so memory is O(K) and it can emit results as it goes. Sorted order is what makes streaming possible.

FOLLOW-UP 2 What if one list may overlap itself?

The two-pointer merge already works, because it folds every incoming interval into the last output interval. What breaks is only the assumption that each list is sorted by start; if that also fails, sort each list first (O(n log n)) or push everything through a heap.

FOLLOW-UP 3 Do touching intervals merge?

Decide the endpoint rule before coding. For closed intervals [1, 3] and [3, 5] share a point, so they merge (start <= last end). For half-open intervals [1, 3) and [3, 5) they do not overlap but are adjacent; whether to merge adjacent ones is a product choice, so ask.

FOLLOW-UP 4 Each list may overlap itself or be unsorted.

Normalise each list first (sort and merge it by itself, O(n log n)), after which the two-pointer union applies. Or sort everything together if memory allows.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
import heapq


# FOLLOW-UP 1: What if the lists are too big for memory?
def union_stream(lists, touching=True):
    # lists: sorted iterables of (start, end)
    merged = heapq.merge(*lists)
    cur = None
    for start, end in merged:
        if cur is None:
            cur = [start, end]
            continue
        # touching: [1, 3] and [3, 5] share the point 3
        joined = start < cur[1]
        if touching:
            joined = start <= cur[1]
        if joined:
            cur[1] = max(cur[1], end)
        else:
            # finished one block: emit it as we go
            yield tuple(cur)
            cur = [start, end]
    if cur is not None:
        yield tuple(cur)


# BASE QUESTION (optimal): Two-pointer merge
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))


# VARIANT: K lists with a heap
def union_k(lists, touching=True):
    # same code, any number of lists, O(K) memory
    return list(union_stream(lists, touching))


# FOLLOW-UP 2: What if one list may overlap itself?
def normalise(intervals):
    # a list that is unsorted or overlaps itself
    ordered = sorted(intervals)
    return list(union_stream([ordered]))


# FOLLOW-UP 4: Each list may overlap itself or be
# unsorted.
def union_unsorted(a, b):
    # fix each list first, then use the same union
    return union_two(normalise(a), normalise(b))


if __name__ == "__main__":
    a = [(1, 3), (6, 8)]
    b = [(2, 4), (9, 10)]
    assert union_two(a, b) == [(1, 4), (6, 8), (9, 10)]
    # closed intervals that touch merge
    assert union_two([(1, 3)], [(3, 5)]) == [(1, 5)]
    # half-open intervals that only touch stay apart
    assert union_two([(1, 3)], [(3, 5)], False) == [
        (1, 3), (3, 5)]
    # three lists, one of them empty
    three = union_k([a, b, []])
    assert three == [(1, 4), (6, 8), (9, 10)]
    # a list that overlaps itself, and an unsorted one
    merged = union_unsorted([(5, 9), (1, 6)], [(8, 12)])
    assert merged == [(1, 12)]
    assert union_unsorted([(9, 10), (1, 2)], []) == [
        (1, 2), (9, 10)]
```

NOTES FROM CANDIDATES

The round intentionally layers one constraint at a time. Keep the base invariant explicit—every interval before the current pointer has already been absorbed into a canonical output prefix—so the streaming and K-way versions remain easy to justify.
Agree on endpoint semantics before coding: touching closed intervals merge, while touching half-open intervals do not overlap. For K streaming inputs, the heap plus one current interval per input uses O(K) working memory, excluding the output.

PREPARATION

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Heaps, Stacks, Sorting and Binary Search, Greedy and Math (listed there as SKIP, no pages). Site tags: array, binary-search, complexity-analysis, medium

MEDIUM | 1Point3Acres | onsite-coding | 8 of 19

Cutting Ribbons (LC 1891)

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 147/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1891 Cutting Ribbons

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/cutting-ribbons-max-equal-length

QUESTION

LC 1891 Cutting Ribbons: given wood lengths and K, maximize the equal piece length that yields at least K pieces. Follow-ups cover the search bound, floating-point lengths, and why complexity depends on the value range rather than the number of elements.SWEarraybinary-searchcomplexity-analysismedium FrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether piece lengths must be integers, what to return if even length 1 fails (0), and the sizes of the array and k.

BRUTE FORCE TO SAY OUT LOUD

Try every length from the maximum down to 1 and count pieces. O(n * max).

OPTIMAL IDEA

Feasibility is monotone: if you can cut k pieces of length L you can for any smaller length. Binary search L over [1, max]: count = sum(r // L), move right if count >= k. O(n log max). A tighter upper bound is min(max, sum // k).

DRY-RUN THESE CASES

k larger than the total length (0), one ribbon, all equal ribbons, k = 1 (answer is the max ribbon).

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Real-valued lengths (fixed number of halvings, not an epsilon loop), why complexity depends on the value range, or an unlimited number of cuts.

PITFALLS

Overflow when summing. Searching from 0 and dividing by zero.

HOW TO THINK ABOUT IT

'Largest length such that at least K pieces fit' is a maximisation with a yes/no check. Larger lengths yield fewer pieces, so the check is monotone and you can binary search the answer. The check itself is one pass: sum of ribbon // length.

Basis: LeetCode 1891, Cutting Ribbons.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        pieces = sum(r // L for r in ribbons)
        if pieces >= k:
            return L
    return 0
```

TIME	SPACE
O(n * max)	O(1)

WHY IT IS SLOW

Tries up to max(ribbon) lengths, each costing O(n); with ribbons up to 10^9 that is a billion probes. The check is monotone, so binary search needs only log2(max) probes.

OPTIMAL SOLUTION · Binary search the answer (integer lengths)

TIME	SPACE
O(n log(max length))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the running time depend on the value range and not just on the number of ribbons?

Each probe costs O(n), and the number of probes is log2(max length) for the integer version. A huge length like 10^9 needs about 30 probes while 10 needs 4, even with the same n. For real numbers the probe count is set by the precision you promise, e.g. log2(max / epsilon).

FOLLOW-UP 2 What if the candidate length is given to you?

Then it is just the feasibility check: sum(r // L for r in ribbons) >= k, in O(n). The binary search exists only to find the best L; the check is the same function you reuse inside it.

FOLLOW-UP 3 What is a good upper bound for the search?

min(max(ribbons), sum(ribbons) // k). If the total length divided by k is smaller than the longest ribbon, no piece longer than that can ever give k pieces, which shrinks the range a little and avoids wasted probes. **Cost:** each probe is O(n); the tighter bound removes only a few of the log2(max) probes, so the total stays O(n log max).

FOLLOW-UP 4 Give the search a tighter upper bound.

No piece can be longer than the longest ribbon, and K pieces of length L need K * L total length, so L <= sum // K. Starting hi at min(max(ribbons), sum // K) saves a few probes and also returns 0 immediately when sum < K.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
def pieces(ribbons, length):
    # how many pieces of this length can we cut?
    return sum(r // length for r in ribbons)


# FOLLOW-UP 4: Give the search a tighter upper bound.
def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and
    # k pieces of length L need k * L total length
    return min(max(ribbons), sum(ribbons) // k)


# BASE QUESTION (optimal): Binary search the answer
# (integer lengths)
def max_length(ribbons, k):
    # the best whole-number length (0 if impossible)
    lo, hi, best = 1, upper_bound(ribbons, k), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if pieces(ribbons, mid) >= k:
            # mid works: remember it and try longer
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best


# VARIANT: fractional lengths
def max_length_float(ribbons, k, rounds=50):
    # lengths may be fractions: fixed rounds of halving
    lo, hi = 0.0, float(max(ribbons))
    for _ in range(rounds):
        mid = (lo + hi) / 2
        # enough pieces: the answer is at least mid
        if sum(int(r // mid) for r in ribbons) >= k:
            lo = mid
        else:
            hi = mid
    return lo


if __name__ == "__main__":
    assert max_length([9, 7, 5], 3) == 5
    assert max_length([7, 5, 9], 4) == 4
    # not enough total length: no pieces at all
    assert max_length([1, 2], 10) == 0
    # the check alone, when the length is given
    assert pieces([9, 7, 5], 5) == 3
    assert upper_bound([9, 7, 5], 3) == 7
    # fractional lengths: 3 pieces from one ribbon of 1
    assert abs(max_length_float([1], 3) - 1 / 3) < 1e-9
    assert max_length_float([1], 0 + 2) > 0.49
```

NOTES FROM CANDIDATES

The interviewer allowed the candidate to abandon an initial greedy/sorting direction, then unlocked the problem with one question: what changes if a candidate length is supplied? Expect the follow-ups, not the base implementation, to consume most of the round.
For the integer version, initialize the best answer to 0, search only positive candidate lengths, and return 0 when even length 1 cannot produce K pieces. The feasibility predicate is monotone: once a length is infeasible, every larger length is also infeasible.

PREPARATION

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Strings, Parsing and Recursion, Dynamic Programming, Greedy and Math (listed there as SKIP, no pages). Site tags: array, backtracking, dp, math, medium

MEDIUM | 1Point3Acres | onsite-coding | 9 of 19

Target Expressions from Ordered Digits

ASKED 2026-08-13 | 68 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 36.5 | 30 DAYS 0 | PRIORITY 103/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 282 Expression Add Operators

Frequency: Single report | Last asked: 2026-08-13 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/target-expressions-ordered-digits

QUESTION

EXAMPLES

```
For digits [2, 3, 4] and target 14, valid outputs
    include:
2 * (3 + 4)
2 + 3 * 4
```

APPROACH

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether digits may be concatenated into multi-digit numbers, which operators are allowed, whether parentheses are free, whether the digit order is fixed, and whether to return all expressions or a boolean.

BRUTE FORCE TO SAY OUT LOUD

Enumerate every operator placement and every parenthesization for the three or four digits and evaluate each. That is fine for a handful of digits and is what the first stage asks for.

OPTIMAL IDEA

Interval DP: for each interval of digits compute a map from value to the expressions that achieve it by splitting at every point and combining left and right with plus, minus and times. Memoize by interval. If only reachability is needed store just the set of values per interval, which is much smaller than the strings.

DRY-RUN THESE CASES

[2, 3, 4] with target 14 gives 2*(3+4) and 2+3*4. A target that is unreachable. A single digit. Zeros.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

N digits (the DP above), division with fractions, or concatenation.

PITFALLS

Missing parentheses in the output string. Evaluating with floats.

HOW TO THINK ABOUT IT

Any expression over a run of digits is two smaller expressions joined by an operator at some split point, so the set of values for an interval is built from the sets of its two halves. Compute values bottom-up per interval and only build text for the target. Be explicit about parentheses: they come from the split structure, so every split is a candidate.

Basis: Related to LeetCode 282, Expression Add Operators, but with parentheses allowed and digits not concatenated.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def values(digits):
    if len(digits) == 1:
        return {digits[0]}
    result = set()
    for cut in range(1, len(digits)):
        for a in values(digits[:cut]):
            for b in values(digits[cut:]):
                result |= {a + b, a - b, a * b}
    return result
reachable = target in values(all_digits)
```

TIME	SPACE
exponential (Catalan(n) *...	**O(n)**
exponential (Catalan(n) * 3^(n-1) expressions)	recursion plus the value set

WHY IT IS SLOW

The same sub-interval of digits is recomputed from scratch for every split above it, giving Catalan-number growth. Remembering the values per interval (memoisation) makes it polynomial in the number of distinct values.

OPTIMAL SOLUTION · Interval DP: all expressions for digits[l:r]

TIME	SPACE
exponential in the number...	**O(distinct values per interval)**
exponential in the number of results; polynomial in the number of distinct values	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you handle N digits where listing every expression is too much?

Agree on the contract: if you only need to know whether a target is reachable, store just the set of values per interval (a boolean DP over (l, r, value)), which is far smaller than the set of strings. If you need one witness, store one expression string per value. Values are bounded by the digits, so the table stays practical for moderate N.

FOLLOW-UP 2 What if concatenating digits (23) or division is allowed?

Concatenation adds base cases: for every l and r the number int(digits[l:r]) is also an atom of the interval. Division adds a fourth operator with a zero check and, if exactness matters, Fraction values instead of floats.

FOLLOW-UP 3 Only a yes/no answer is needed for many digits.

Store just the set of reachable values per interval (as target_reachable does) instead of the expression strings; the strings blow up long before the value sets do. Build the expression only for the winning target by remembering one split per value.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict


def build(digits):
    # table[(l, r)] = {value: {(text, precedence)}}
    table = {}

    def solve(l, r):
        if (l, r) in table:
            return table[(l, r)]
        res = defaultdict(set)
        if r - l == 1:
            # a single digit is an atom (precedence 3)
            res[digits[l]].add((str(digits[l]), 3))
        # try every split point
        for m in range(l + 1, r):
            for lv, ls in solve(l, m).items():
                for rv, rs in solve(m, r).items():
                    combine(res, lv, ls, rv, rs)
        table[(l, r)] = res
        return res

    return solve(0, len(digits))


def combine(res, lv, ls, rv, rs):
    ops = (("+", 1, lv + rv), ("-", 1, lv - rv),
           ("*", 2, lv * rv))
    for lt, lp in ls:
        for rt, rp in rs:
            for op, prec, val in ops:
                # brackets only where precedence needs
                left = f"({lt})" if lp < prec else lt
                need = rp < prec
                if rp == prec and op == "-":
                    need = True
                right = f"({rt})" if need else rt
                res[val].add((left + op + right, prec))


# BASE QUESTION (optimal): Interval DP: all expressions
# for digits[l:r]
def expressions_for_target(digits, target):
    found = build(digits).get(target, ())
    return sorted({text for text, _ in found})
```

```
# FOLLOW-UP 3: Only a yes/no answer is needed for many
# digits.
def target_reachable(digits, target):
    # yes/no only: keep just the set of values per range
    values = {}

    def solve(l, r):
        if (l, r) in values:
            return values[(l, r)]
        out = {digits[l]} if r - l == 1 else set()
        for m in range(l + 1, r):
            for a in solve(l, m):
                for b in solve(m, r):
                    out |= {a + b, a - b, a * b}
        values[(l, r)] = out
        return out

    return target in solve(0, len(digits))


if __name__ == "__main__":
    ans = expressions_for_target([2, 3, 4], 14)
    assert "2*(3+4)" in ans and "2+3*4" in ans
    assert target_reachable([2, 3, 4], 14)
    assert not target_reachable([2, 3, 4], 100)
    # subtraction needs brackets on the right
    sub = expressions_for_target([5, 3, 1], 3)
    assert "5-(3-1)" in sub
    assert target_reachable([1] * 8, 8)
```

NOTES FROM CANDIDATES

The base case is small enough for direct enumeration, but a complicated ad hoc implementation becomes difficult to extend. Keep expression construction separate from numeric evaluation while negotiating the N-digit contract.

If the agreed contract uses every digit exactly once in order, memoize each interval [l, r) as a map from attainable numeric values to the expression strings producing them. For every split point, combine left and right entries with + and *, wrapping composite children as needed; the full-interval target bucket is the answer. Because every distinct expression must be retained, both time and storage are output-sensitive and can grow exponentially even though interval states are memoized.

PREPARATION

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Heaps, Stacks, Sorting and Binary Search (listed there as SKIP, no pages). Site tags: array, hashmap, sorting, medium

MEDIUM | 1Point3Acres | phone-screen | 10 of 19

Progressive Array Partitioning

ASKED 2026-08-11 | 70 DAYS AGO | LAST 90 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 198/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-11 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/array-partition-unique-frequencies-straights

QUESTION

EXAMPLES

Stage 1

```
[1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true because the
    four value-groups have different sizes.
[1, 2, 3] -> false because every value-group has size
    one.
```

Stage 2

```
[1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true.
[4, 5, 6, 7, 7, 8, 9] -> false.
```

APPROACH

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the groups are and the rule for each stage. From the examples, the first stage looks like: true when the counts of all distinct values are different from each other (1:1, 2:2, 3:3, 4:4 is true; all counts 1 is false). The second stage is not clear from the examples alone, so ask for its exact rule and test it on both examples before coding.

BRUTE FORCE TO SAY OUT LOUD

Try every way to group the elements and test the rule. Exponential.

OPTIMAL IDEA

For stage one: count each value with a hash map, then check that the multiset of counts has no repeats (a set of counts has the same size as the number of values). O(n). Keep the data model (a counter) easy to adapt, because the second stage arrives only after the first is written.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

The four examples in the statement, an empty array, a single value, many values with equal counts.

LIKELY FOLLOW-UPS

Stage two, which changes the grouping rule; keep helper functions small so it is a local change.

PITFALLS

Coding stage two from a guess. Not reconciling the rule with both examples.

HOW TO THINK ABOUT IT

The statement arrives in stages, so keep the data model easy to adapt: count the values first, because any rule about 'groups of equal values' is a rule about the counts. Stage 1 is then 'are all counts different', which is a set-size comparison. Before stage 2, reconcile the new rule with every example; here a natural guess (LeetCode 659) contradicts the second example, which is exactly the situation where you ask.

Basis: Written. Stage 1 is inferred from the examples: true when every distinct value occurs a different number of times. The Stage 2 rule is NOT recoverable from the two examples, so only a candidate reading is shown; ask the interviewer for the exact rule and test it against both examples before coding.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    counts = Counter(a)
    for x, y in combinations(counts, 2):
        if counts[x] == counts[y]:
            return False
    return True
```

TIME	SPACE
O(d^2) where d is the number of distinct values	O(d)

WHY IT IS SLOW

O(d^2) pair checks for d distinct values. Comparing the number of distinct counts with the number of distinct values (a set) is O(n).

OPTIMAL SOLUTION · Counter, then compare the set of counts with the number of values

TIME	SPACE
O(n)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

FOLLOW-UP 2 Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
from collections import Counter


def counts_of(nums):
    # every rule about groups of equal values is a
    # rule about these counts, so compute them once
    return Counter(nums)


# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of ...
def stage1(counts):
    # true when every value occurs a different number of
    # times: the counts have no repeats
    return len(set(counts.values())) == len(counts)


# FOLLOW-UP 1: Stage 2 arrives. What do you do?
def stage2_candidate(nums, min_len=3):
    # ONE reading of stage 2 (LeetCode 659): split the
    # array into runs of consecutive values, each at
    # least min_len long. Check it against BOTH examples.
    left, ends = Counter(nums), Counter()
    for x in nums:
        if left[x] == 0:
            continue
        left[x] -= 1
        if ends[x - 1] > 0:
            # extend a run that ended at x - 1
            ends[x - 1] -= 1
            ends[x] += 1
        elif left[x + 1] > 0 and left[x + 2] > 0:
            # start a new run x, x + 1, x + 2
            left[x + 1] -= 1
            left[x + 2] -= 1
            ends[x + 2] += 1
        else:
            return False
    return True


# each stage is one small function, so a new rule
# replaces one entry instead of the whole pipeline
RULES = {1: lambda nums: stage1(counts_of(nums)),
         2: stage2_candidate}


def check(stage, nums):
    return RULES[stage](nums)


if __name__ == "__main__":
    # stage 1 examples
    assert check(1, [1, 2, 3, 4, 2, 3, 3, 4, 4, 4])
    assert not check(1, [1, 2, 3])
    # stage 2 example that the candidate gets right
    assert check(2, [1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
    # the second stage-2 example says False, but this
    # reading says True: the rule is incomplete, so ask
    assert check(2, [4, 5, 6, 7, 7, 8, 9]) is True
```

NOTES FROM CANDIDATES

The interviewer presents Stage 2 only after the first implementation, so keep the initial data model easy to adapt.
Before coding the straight variant, reconcile the length rule with both examples; otherwise a superficially valid decomposition can contradict the expected result.
Evaluation emphasized the efficiency of the chosen data structures, not only whether the sample cases passed.
Solution skeleton
Stage 1: count each distinct value, then verify that the frequency values are themselves unique. A hash map plus a set gives O(n) time and O(u) space for u distinct values.
Stage 2: first pin down a group size k. For the fixed-size interpretation, reject when n % k != 0, process keys in ascending order, and whenever the smallest remaining key x has multiplicity c, remove c copies from every key in [x, x + k). Any missing count makes the partition impossible. Sorting the u keys gives O(n + u log u) time and O(u) space.
Do not let length-one groups make every input trivially valid unless the interviewer explicitly allows them; the negative example requires a stronger group-size contract.

PREPARATION

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: array, simulation, sliding-window, medium

MEDIUM | 1Point3Acres | onsite-coding | 11 of 19

Piano Hand Movement Grouping

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 205/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/piano-hand-movement-grouping

QUESTION

EXAMPLES

```
With notes = [1, 3, 5, 7, 9, 11] and k = 4, the
    stated output is 2.
Earlier examples use [1,2,3,4,5] -> 0 and [5,9,1] ->
    1, without stating the reach parameter.
```

APPROACH

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

INTERVIEW-DAY PLAN

CLARIFY FIRST

The statement is ambiguous, so ask: does the hand cover a fixed interval of keys [p, p + k], is the interval fixed until the hand lifts, does the first placement count as a move, and do notes arrive in the given order. Notice the examples are not all consistent, so ask which one is the reference.

BRUTE FORCE TO SAY OUT LOUD

Try every starting position for each group and take the minimum. Exponential or quadratic.

OPTIMAL IDEA

Greedy one-pass scan: start a group at the first note with the hand covering [note, note + k]. For each next note, if it is inside the interval keep going; if not, start a new group at that note. The answer is the number of groups (or groups minus one if the first placement is free). Group boundaries come from the same scan. O(n).

DRY-RUN THESE CASES

A single note, the example [1, 3, 5, 7, 9, 11] with k = 4, notes going down and then up, k = 0.

LIKELY FOLLOW-UPS

A hand that can slide, two hands, or return the group boundaries.

INTERVIEW-DAY PLAN

PITFALLS

Counting the initial placement inconsistently. Assuming notes are sorted.

HOW TO THINK ABOUT IT

Turn the physical rule into an invariant: the current group is valid while (largest note - smallest note) <= k, because you can place the hand at the smallest. The first note that breaks it must start a new group, and starting later never helps, so a single greedy pass is optimal. Ask whether the first placement counts as a move.

Basis: Written; interval grouping, like LeetCode 452 and 1326. Assumed rule (confirm it, the examples disagree): a hand placed on key p covers p to p + k inclusive and stays put until it is lifted; the notes come in order; the answer is the minimum number of placements, and the number of lifts is one fewer.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = [INF] * (n + 1)
best[0] = 0
for j in range(1, n + 1):
    for i in range(j):
        part = notes[i:j]
        if max(part) - min(part) <= k:
            best[j] = min(best[j], best[i] + 1)
answer = best[n]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every end tries every start and re-checks the range, O(n^2). A greedy pass that starts a new group only when the next note no longer fits is O(n).

OPTIMAL SOLUTION · Greedy scan, tracking the group's smallest and largest note

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where each group starts and where to place the hand.

Record the start index when a group opens and emit (start, end, smallest note) when it closes. The state transition is unchanged.

FOLLOW-UP 2 Count lifts, not placements.

Lifts are placements minus one if the first placement is free; the examples differ on this, so ask and state it. In code it is a single subtraction on the group count.

FOLLOW-UP 3 The hand may slide without a lift (costing something).

Then it becomes a DP over positions with a cost per slide distance, solved with the O(n * range) DP or by greedy if the cost is monotone. Ask for the cost model.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
def hand_positions(notes, k):
    # split the notes into groups that one hand position
    # (a span of at most k) can play, as early as possible
    groups = []
    for i, x in enumerate(notes):
        if groups:
            start, lo, hi = groups[-1]
            # does the note fit the current span?
            lo2, hi2 = min(lo, x), max(hi, x)
            if hi2 - lo2 <= k:
                groups[-1] = (start, lo2, hi2)
                continue
        # otherwise the hand must move: a new group starts
        groups.append((i, x, x))
    return groups


# BASE QUESTION (optimal): Greedy scan, tracking the
# group's smallest and largest note
def groups(notes, k):
    # how many hand placements are needed
    return len(hand_positions(notes, k))


# FOLLOW-UP 2: Count lifts, not placements.
def lifts(notes, k):
    # the first placement is free; every move is a lift
    return max(groups(notes, k) - 1, 0)


# FOLLOW-UP 1: Return where each group starts and where to
# place the hand.
def plan(notes, k):
    # where each group starts and ends, and where to put
    # the hand (on the lowest note of the group)
    out, spots = [], hand_positions(notes, k)
    for n, (start, lo, hi) in enumerate(spots):
        end = spots[n + 1][0] - 1 if n + 1 < len(spots) \
            else len(notes) - 1
        out.append((start, end, lo))
    return out


if __name__ == "__main__":
    # the stated example: 2 placements for reach 4
    assert groups([1, 3, 5, 7, 9, 11], 4) == 2
    # the earlier examples, counted as lifts
    assert lifts([1, 2, 3, 4, 5], 4) == 0
    assert lifts([5, 9, 1], 4) == 1
    assert groups([], 4) == 0
    assert plan([1, 3, 5, 7, 9, 11], 4) == [(0, 2, 1),
                                            (3, 5, 7)]
    # a reach of 0 means a new placement for every change
    assert groups([2, 2, 3], 0) == 2
```

NOTES FROM CANDIDATES

Clarify whether a placement is fixed until the hand lifts, whether the covered interval may slide without counting as a lift, and whether the initial placement counts. These details materially change the answer.
The six-note example's explanation says a placement at 7 covers keys 7–11 but still counts another lift for 9 and 11. Treat that as an ambiguity to resolve before coding rather than silently assuming the output convention.
Maintain the feasible placement interval for the current group. The follow-up printout should reuse the same grouping state instead of duplicating the movement logic.

PREPARATION

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: python, hashmap, data-analysis, sql, medium

MEDIUM | 1Point3Acres | phone-screen | 12 of 19

Website Activity Analytics: Unique Users + Session Time

ASKED 2026-06-26 | 116 DAYS AGO | LAST 6 MONTHS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 217/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-26 | Roles: DS

Original: https://www.1point3acres.com/interview/problems/company/google/website-activity-analytics-unique-users-session-time

QUESTION

Given website action records with **`user_id`**, timestamp, and activity, compute per-activity unique-user counts and average time spent on the site per user in seconds. The screen paired this Python task with a lighter SQL section and emphasized edge-case discussion over perfect final code.

Input is a collection of user-activity records. Each record contains:

user_id: numeric user identifier
timestamp: timestamp string such as 2024-07-26 10:00:00
activity: action name such as login, view, purchase, or logout

Implement two analytics tasks

Return a mapping from each activity type to the number of unique users who performed that activity.
Calculate the average time spent on the website per user in seconds.

Clarify session semantics before coding. The prompt uses login/logout-style events, but the exact treatment of users without an explicit logout, multiple sessions per user, and out-of-order records must be agreed with the interviewer.

EXAMPLES

```
user_activity = [
    {'user_id': 1, 'timestamp': '2024-07-26
        10:00:00', 'activity': 'login'},
    {'user_id': 2, 'timestamp': '2024-07-26
        10:05:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:10:00', 'activity': 'view'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:15:00', 'activity': 'purchase'},
    {'user_id': 2, 'timestamp': '2024-07-26
        10:20:00', 'activity': 'view'},
    {'user_id': 3, 'timestamp': '2024-07-26
        10:25:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:30:00', 'activity': 'logout'},
]
```

APPROACH

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

HOW TO THINK ABOUT IT

Unique users per activity is a map from activity to a set of user ids. Time on site needs a definition: pair each login with the next logout of the same user, and say what you do for a login without a logout (here: the session ends at that user's last event). Parse timestamps once, sort each user's events by time, and agree on out-of-order rows and repeated logins before coding.

Basis: Written for this question (a grouping and aggregation task; the SQL half of the screen is the same idea with GROUP BY and COUNT(DISTINCT)).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for activity in activities:
    users = set()
    for rec in records:
        if rec.activity == activity:
            users.add(rec.user)
    unique[activity] = len(users)
for user in all_users:
    mine = [e for e in records if e.user == user]
    events = sorted(mine)
    spent = 0
    for login, logout in pairs(events):
        spent += logout.time - login.time
    total[user] = spent
average = sum(total.values()) / len(total)
```

TIME	SPACE
O(U * N + A * N)	O(U + A)

WHY IT IS SLOW

Rescans the whole log for every activity and every user. One pass that groups by activity and by user, then sorts each user's events once, is O(N log N).

OPTIMAL SOLUTION · One pass to group, then sessions per user

TIME	SPACE
O(N log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Users with several sessions, missing logouts or out-of-order rows.

Sort each user's events by time before pairing; a second login before a logout is ignored (or closes the old session, depending on the contract you agree). A login with no logout ends at the user's last event, or is dropped if you decide it is unreliable. Say the choice out loud: the interviewer accepted a strong discussion over perfect code.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict
from datetime import datetime

FORMAT = "%Y-%m-%d %H:%M:%S"


def when(record):
    # timestamp text -> datetime
    return datetime.strptime(record["timestamp"], FORMAT)


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
def unique_users_by_activity(records):
    users = defaultdict(set)
    for r in records:
        # a set counts every user once per activity
        users[r["activity"]].add(r["user_id"])
    return {a: len(u) for a, u in sorted(users.items())}


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# FOLLOW-UP 1: Users with several sessions, missing
# logouts or out-of-order rows.
def session_seconds(events):
    # events: one user's (time, activity), any order
    total, start = 0, None
    # FOLLOW-UP 1: sorted: rows may arrive out of order
    for t, act in sorted(events):
        if act == "login" and start is None:
            # a second login before a logout is ignored
            start = t
        elif act == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    # a login with no logout ends at the last event seen
    if start is not None:
        last = max(t for t, _ in events)
        total += (last - start).total_seconds()
    return total


# AVERAGE time on the site per user
def average_time_per_user(records):
```

```
    events = defaultdict(list)
    for r in records:
        item = (when(r), r["activity"])
        events[r["user_id"]].append(item)
    if not events:
        return 0.0
    # state the denominator: every user who appears
    per_user = events.values()
    total = sum(session_seconds(e) for e in per_user)
    return total / len(events)


if __name__ == "__main__":
    def rec(user, minute, activity):
        stamp = f"2024-07-26 10:{minute:02}:00"
        return {"user_id": user, "activity": activity,
                "timestamp": stamp}

    data = [rec(1, 0, "login"), rec(2, 5, "login"),
            rec(1, 10, "view"), rec(1, 15, "purchase"),
            rec(2, 20, "view"), rec(3, 25, "login"),
            rec(1, 30, "logout")]
    assert unique_users_by_activity(data) == {
        "login": 3, "logout": 1, "purchase": 1, "view": 2}
    # user 1: 1800 s, user 2: 900 s (no logout), user 3: 0
    assert average_time_per_user(data) == 900
    assert average_time_per_user([]) == 0.0
    # out-of-order rows are sorted first
    shuffled = [rec(1, 30, "logout"), rec(1, 0, "login")]
    assert average_time_per_user(shuffled) == 1800
    # two sessions add up
    two = [rec(1, 0, "login"), rec(1, 5, "logout"),
           rec(1, 10, "login"), rec(1, 20, "logout")]
    assert average_time_per_user(two) == 900
```

NOTES FROM CANDIDATES

SQL in the same screen was described as simple and covered joins / aggregation style work, but the exact SQL prompt did not leak.
The interviewer accepted a strong draft when all edge cases were discussed, even though the final code did not cover every case.
Spend time up front on timestamp parsing, ordering, repeated login/logout events, and users with missing terminal actions.

PREPARATION

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Site tags: array, hashmap, medium

MEDIUM | 1Point3Acres | onsite-coding | 13 of 19

Gold Chain Split into Two Equal Halves

ASKED 2026-04-21 | 182 DAYS AGO | OLDER

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 201/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-21 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gold-chain-split-equal

QUESTION

Onsite coding interview 1: given a gold chain represented by an array of segment weights, remove exactly one segment and check whether the remaining segments — when reconnected — can be cut once more to give two halves of equal total weight. Follow-up: enumerate all valid (remove, cut) pairs.SWEArrayhashmapmediumFrequencySingle reportLast asked2026-04-21Stageonsite-coding

APPROACH

Drill The equal-subset-sum and partition-to-k-equal-sum-subsets problems are the standard drill partners.
Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.
Watch the boundary: remove + cut should not yield an empty side.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether removing a segment keeps the rest as a chain joined end to end, whether the cut after removal must leave two non-empty parts, and whether to return a boolean, one answer, or all valid pairs.

BRUTE FORCE TO SAY OUT LOUD

For each removed index, rebuild the array and try every cut with prefix sums. O(n^2).

OPTIMAL IDEA

Compute the total S and prefix sums P. Removing index r leaves total S - a[r], which must be even; the target is half = (S - a[r]) / 2. In the remaining chain a cut is valid when some non-empty prefix sums to half. A prefix that stops before r has sum P[j], and one that includes the segments after r has sum P[j] - a[r]. So for each r check whether half appears among the prefix sums before r, or whether half + a[r] appears among the prefix sums after r (keep a hash set of prefix sums seen so far, and one for those still ahead). Boolean answer in O(n); listing all valid (remove, cut) pairs is output-bound, up to O(n^2).

DRY-RUN THESE CASES

A chain of two segments, a removal at the first or last index, a total that becomes odd, equal weights.

LIKELY FOLLOW-UPS

Enumerate all valid (remove, cut) pairs, or allow removing k segments.

PITFALLS

Allowing an empty half. Forgetting the shift of indices after the removed segment.

INTERVIEW-DAY PLAN

Compute the total S and prefix sums P. Removing index r leaves total S - a[r], which must be even; the target is half = (S - a[r]) / 2. In the remaining chain a cut is valid when some non-empty prefix sums to half. A prefix that stops before r has sum P[j], and one that includes the segments after r has sum P[j] - a[r]. So for each r check whether half appears among the prefix sums before r, or whether half + a[r] appears among the prefix sums after r (keep a hash set of prefix sums seen so far, and one for those still ahead). Boolean answer in O(n); listing all valid (remove, cut) pairs is output-bound, up to O(n^2).

DRY-RUN THESE CASES

A chain of two segments, a removal at the first or last index, a total that becomes odd, equal weights.

LIKELY FOLLOW-UPS

Enumerate all valid (remove, cut) pairs, or allow removing k segments.

PITFALLS

Allowing an empty half. Forgetting the shift of indices after the removed segment.

HOW TO THINK ABOUT IT

After removing segment r the target half is (S - a[r]) / 2, and a cut means some prefix of the remaining chain has exactly that weight. A prefix of the remaining chain is either an ordinary prefix of the original array (stops before r) or an original prefix minus a[r] (passes over r). Each is one value to look up in a hash of prefix sums, so every r costs O(1).

Basis: Written; prefix sums with a hash lookup (like LeetCode 724 and 560). Assumed rule (confirm it): remove exactly one segment, reconnect the remaining chain end to end, then cut it once into two non-empty parts of equal total weight.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(w):
    for r in range(len(w)):
        chain = w[:r] + w[r + 1:]
        half, run = sum(chain) / 2, 0
        for x in chain:
            run += x
            if run == half:
                return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n removals, each rebuilt and scanned: O(n^2). Prefix sums with a lookup of the needed prefix value answer each removal in O(1).

OPTIMAL SOLUTION · Prefix sums with first and last occurrence maps

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 List every valid (removed segment, cut position) pair.

The answer itself can be quadratic, so a simple loop is acceptable. With positive weights prefix sums are strictly increasing, so each target has at most one cut position and the listing is O(n) after the hash lookups; with zeros or negatives a value can repeat, so keep a list of indices per prefix value.

FOLLOW-UP 2 The chain is a circle (cut anywhere).

Then a cut is two cut points on the circle and the removed segment just shortens it; use the circular window idea (duplicate the array and slide a window whose sum is half).

FOLLOW-UP 3 Remove k segments instead of one.

The remaining total depends on which k you remove, which becomes a subset-sum style search; ask for the limits before promising a fast solution.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
def prefix_sums(a):
    # P[j] = sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[-1] + x)
    return P


# BASE QUESTION (optimal): Prefix sums with first and last
# occurrence maps
def gold_chain(a):
    # remove exactly one segment, then cut once into
    # two non-empty halves of equal weight
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[-1]
    # first and last position of every prefix sum (cuts
    # sit between segments, so positions 1..n-1)
    first, last = {}, {}
    for j in range(1, n):
        first.setdefault(P[j], j)
        last[P[j]] = j
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2:
            continue
        half = left_over // 2
        # cut before the removed segment: P[j] == half
        if half in first and first[half] <= min(r, n - 2):
            return True
        # cut after it: P[j] - a[r] == half
        if half + a[r] in last and last[half + a[r]] > r:
            return True
    return False


# FOLLOW-UP 1: List every valid (removed segment, cut
# position) pair.
def valid_pairs(a):
    # follow-up: every (removed index, cut position) pair;
    # the answer can be quadratic, so a loop is fine
    pairs = []
    for r in range(len(a)):
        rest = a[:r] + a[r + 1:]
        total, run = sum(rest), 0
        # cut after `cut` segments of the reconnected
        # chain
        for cut in range(1, len(rest)):
            run += rest[cut - 1]
            if run * 2 == total:
                pairs.append((r, cut))
    return pairs


if __name__ == "__main__":
    # remove the middle 1, then cut 5 | 5
    assert gold_chain([5, 1, 5])
    assert gold_chain([2, 7, 2])
    assert not gold_chain([1, 2, 3])
    assert not gold_chain([1, 1])
    assert not gold_chain([1, 2, 4])
    assert (1, 1) in valid_pairs([1, 2, 1])
    # both methods agree on many small chains
    import itertools
    for chain in itertools.product(range(1, 5), repeat=5):
        assert gold_chain(list(chain)) == bool(
            valid_pairs(list(chain)))
```

NOTES FROM CANDIDATES

Precompute prefix sums; for each candidate removed index r, the remaining total is S - a[r]. If that is odd, skip. Otherwise check if there is a split point achieving (S - a[r]) / 2.
Use a hashmap from prefix-sum value to index for O(1) lookups -> overall O(N) for the boolean answer, O(N²) worst case for enumerating all pairs (output-bound).
Watch out for cut positions adjacent to the removed link — must still respect contiguous-chain semantics.

PREPARATION

Drill The equal-subset-sum and
partition-to-k-equal-sum-subsets problems are the standard drill partners.
Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.
Watch the boundary: remove + cut should not yield an empty side.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Site tags: array, two-pointer, medium

MEDIUM | 1Point3Acres | onsite-coding | 14 of 19

Sum of Good Subarrays (Adjacent Diff ±1)

ASKED 2026-02-08 | 254 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 208/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-08 | Roles: MLE, SWE

Original: https://www.1point3acres.com/interview/problems/company/google/good-subarrays-diff-one

QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

Input: integer array arr of length N.
A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).
Singleton subarrays are always good.
Output: the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).
Required: O(N) time.

EXAMPLES

```
arr = [3, 5, 6, 7, 6]
```

Good subarrays

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
    -1).
```

APPROACH

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.
Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.
Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

HOW TO THINK ABOUT IT

A good subarray is a stretch where every step is +1 or every step is -1, so every good subarray ending at index i lies inside the current run. Instead of listing subarrays, keep for each direction the total of all suffix sums inside the run: extending a run of length L by a new element x adds (L + 1) * x to that total. Both directions can be updated in O(1), and the singleton is counted once.

Basis: Written for this question (a run-length DP in the style of LeetCode 2262, Total Appeal of A String).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if is_good(part):
                total += sum(part)
    return total
def is_good(part):
    pairs = zip(part, part[1:])
    steps = {y - x for x, y in pairs}
    return steps <= {1} or steps <= {-1}
```

TIME	SPACE
O(n^2) or O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) subarrays (O(n^3) with the sums). Run totals per direction update in O(1) per element, so one pass is O(n).

OPTIMAL SOLUTION · Two running run totals

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the singleton subtracted?

A one-element subarray [x] is both an increasing and a decreasing run of length 1, so it appears in both running totals. Subtracting x once at each index keeps it counted exactly once. A subarray of length 2 or more can only be in one total because its step is +1 or -1, not both.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# BASE QUESTION (optimal): Two running run totals
def good_subarray_sum(a):
    # a good subarray has all steps +1 or all steps -1.
    # For each index keep, per direction, the sum of the
    # sums of all good subarrays that END here.
    total = 0
    up_sum = up_len = down_sum = down_len = 0
    for i, x in enumerate(a):
        step = x - a[i - 1] if i else None
        if step == 1:
            # extend the +1 run: every old subarray grows
            # by
            # x, and [x] alone is new: (len + 1) * x in
            # all
            up_sum += (up_len + 1) * x
            up_len += 1
        else:
            up_sum, up_len = x, 1
        if step == -1:
            down_sum += (down_len + 1) * x
            down_len += 1
        else:
            down_sum, down_len = x, 1
        # [x] is in both totals: count it once
        total += up_sum + down_sum - x
    return total


# BRUTE FORCE (checker)
def good_subarray_sum_brute(a):
    # check every subarray: the oracle for the demo
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            steps = {a[k + 1] - a[k] for k in range(i, j)}
            if steps <= {1} or steps <= {-1}:
                total += sum(a[i:j + 1])
    return total


if __name__ == "__main__":
    arr = [3, 5, 6, 7, 6]
    # singletons 27 + [5,6] 11 + [6,7] 13 + [5,6,7] 18 +
    # [7,6] 13
    assert good_subarray_sum(arr) == 82
```

```python
    assert good_subarray_sum([]) == 0
    assert good_subarray_sum([4]) == 4
    # the O(n) answer matches the brute force everywhere
    import itertools
    for n in range(7):
        every = itertools.product([1, 2, 3, 4], repeat=n)
        for a in every:
            assert good_subarray_sum(list(a)) == (
                good_subarray_sum_brute(list(a)))
```

NOTES FROM CANDIDATES

Walk the array maintaining two run lengths: upRun (current trailing run of +1 steps) and downRun (current trailing run of -1 steps). At index i, only one of them is nonzero (the other resets to 1).

Each index i ends upRun + downRun - 1 good subarrays (subtracting the double-counted singleton). For each ending position, contribute the running prefix-sum tail of the active run.

Multiple candidates report getting stuck — verbalize the "contribution per index" framing out loud before coding.

Easy to mis-handle the reset boundary; double-check with a 2-element example before writing the loop.

PREPARATION

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.

Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.

Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: hashmap, two-pointer, streaming, medium

MEDIUM | 1Point3Acres | onsite-coding | 15 of 19

Two-Day Log Intersection under Memory Limit

ASKED 2025-11-27 | 327 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 46.9 | PRIORITY 24/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 349 Intersection of Two Arrays (related)

Frequency: Single report | Last asked: 2025-11-27 | Roles: SWE, INFRA-ENG

Original: https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

QUESTION

PhD intern coding round 1: two log files with **`(timestamp, obj_id, client_id)`** entries. Find **`obj_id`**s that (a) appear in both files and (b) have at least two distinct **`client_id`**s. Follow-up imposes a memory limit, expecting external sort + two-pointer.

Input: two log files; each line is (timestamp, obj_id, client_id). Find every obj_id such that:

(a) appears in both files, and
(b) is associated with at least 2 distinct client_ids (aggregated across both files).

Follow-up 1: complexity of set-intersection on the file contents.
Follow-up 2: memory cannot hold either file fully — design a streaming solution.

EXAMPLES

```
File A:  (t1, X, c1), (t2, X, c2), (t3, Y, c1)
File B:  (t4, X, c1), (t5, Z, c2)
-> Only X is in both files; X has 2 distinct clients
    {c1, c2}.
Output: [X]
```

APPROACH

Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

HOW TO THINK ABOUT IT

In memory it is a hash map from object to the set of clients, filtered to objects that appear in both files and have at least two clients. When the files do not fit in memory, sort both by object id (external sort) and walk them together: all rows of one object are adjacent, so you only ever hold the clients of one object at a time.

Basis: LeetCode 349, Intersection of Two Arrays, with a distinct-client condition and a limit on memory (external sort and a two-pointer merge).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(first, second):
    result = []
    for obj in {r.obj for r in first}:
        if obj not in {r.obj for r in second}:
            continue
        rows = [r for r in first + second
                if r.obj == obj]
        if len({r.client for r in rows}) >= 2:
            result.append(obj)
    return result
```

TIME	SPACE
O(n * m)	O(n + m)

WHY IT IS SLOW

O(n * m) time and both files must fit in memory. Sorting both files by object and merging them with two pointers is O(n log n) and holds one object's clients at a time.

OPTIMAL SOLUTION · Hash map of client sets (everything fits in memory)

TIME	SPACE
O(n + m)	O(n + m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why prefer sort and two pointers to a streaming hash map?

A hash map needs memory proportional to the number of distinct objects (and clients), which is exactly what the limit forbids. An external sort needs only a chunk of memory and sequential disk passes, and the merge needs memory for one object's clients. If one object has an enormous client set, count distinct clients approximately (HyperLogLog) or stop as soon as two are seen.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict


# BASE QUESTION (optimal): Hash map of client sets
# (everything fits in memory)
def log_objects(day1, day2):
    # everything fits in memory: object -> set of clients
    c1, c2 = defaultdict(set), defaultdict(set)
    for _, obj, client in day1:
        c1[obj].add(client)
    for _, obj, client in day2:
        c2[obj].add(client)
    # in both files, with 2+ clients over both days
    both = c1.keys() & c2.keys()
    return sorted(o for o in both
                  if len(c1[o] | c2[o]) >= 2)


def groups(rows):
    # rows sorted by object: yield (object, its clients)
    current, clients = None, set()
    for _, obj, client in rows:
        if obj != current:
            if current is not None:
                yield current, clients
            current, clients = obj, set()
        clients.add(client)
    if current is not None:
        yield current, clients


# FOLLOW-UP 1: Why prefer sort and two pointers to a
# streaming hash map?
def log_objects_sorted(rows1, rows2):
    # memory limit: both files arrive sorted by object (an
    # external sort), so only one object's clients are
    # held
    out = []
    g1, g2 = groups(rows1), groups(rows2)
    a, b = next(g1, None), next(g2, None)
    while a and b:
        if a[0] == b[0]:
            # same object in both files
            if len(a[1] | b[1]) >= 2:
                out.append(a[0])
            a, b = next(g1, None), next(g2, None)
        elif a[0] < b[0]:
            a = next(g1, None)
        else:
            b = next(g2, None)
    return out


if __name__ == "__main__":
    day1 = [(1, "X", "c1"), (2, "X", "c2"),
            (3, "Y", "c1")]
    day2 = [(4, "X", "c1"), (5, "Z", "c2")]
    assert log_objects(day1, day2) == ["X"]
    def by_object(rows):
        return sorted(rows, key=lambda r: r[1])

    assert log_objects_sorted(by_object(day1),
                              by_object(day2)) == ["X"]
    # one client in total is not enough
    one = [(1, "A", "c1")]
    assert log_objects(one, one) == []
    assert log_objects_sorted(one, one) == []
    # clients may be split across the two days
    d1, d2 = [(1, "A", "c1")], [(2, "A", "c2")]
    assert log_objects(d1, d2) == ["A"]
    assert log_objects_sorted(d1, d2) == ["A"]
    assert log_objects([], day2) == []
```

NOTES FROM CANDIDATES

In-memory solution: build a hashmap obj_id -> set(client_id) while scanning each file; intersect keys across the two hashmaps at the end.
Memory-limited solution: external sort each file by obj_id, then run a merge / two-pointer scan over both sorted files. For each matching obj_id group, count distinct client_id on the fly (HyperLogLog if duplicates are large).
The interviewer explicitly preferred sort + two-pointer over a streaming hashmap — be prepared to defend the choice (sort is O(n log n) on disk vs O(n) RAM-bound for the hashmap).
For distributed scale, hash-partition by obj_id and run the same algorithm per partition.

PREPARATION

Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: array, grid, medium

MEDIUM | 1Point3Acres | onsite-coding | 16 of 19

Rotate Any m×n 2D Matrix

STUDY: TYPE | **ALL TIME 56.3** | **30 DAYS 0** | **PRIORITY 57/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #48 Rotate Image

LeetCode match: LC 48 Rotate Image (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/rotate-matrix-rectangle

QUESTION

EXAMPLES

```
[[1,2,3],[4,5,6]] (2×3) rotated 90° CW ->
    [[4,1],[5,2],[6,3]] (3×2).
```

APPROACH

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which direction and angle, whether in place is required, and the dimensions. A non-square matrix cannot be rotated in place in the same buffer, since its shape changes.

BRUTE FORCE TO SAY OUT LOUD

Create a new matrix and copy each cell to its rotated position with an index formula. O(mn) space.

OPTIMAL IDEA

Clockwise 90 degrees: reverse the rows and transpose, [list(r) for r in zip(*matrix[::-1])]. Counter-clockwise: transpose then reverse the rows. 180 degrees: reverse the rows and each row. For a square matrix rotate in place by cycling four cells per layer.

DRY-RUN THESE CASES

The 2 x 3 example giving [[4,1],[5,2],[6,3]], 1 x 1, 1 x n, an empty matrix, four rotations returning the original.

LIKELY FOLLOW-UPS

In place square rotation, rotation by k times 90 degrees, or non-square in place with a flat array and cycles.

PITFALLS

Mixing up the index formula direction. Trying to rotate a rectangle in place in a 2D list.

HOW TO THINK ABOUT IT

A 90 degree clockwise rotation sends the cell at row i, column j to row j, column (m - 1 - i) of the new n x m matrix. That formula is the whole problem; 'reverse the rows then transpose' is just a short way to write it. A rectangle cannot be rotated in place because its shape changes.

Basis: Generalizes LeetCode 48, Rotate Image, from an n x n matrix to any m x n matrix.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def rotate(old):
    m, n = len(old), len(old[0])
    new = [[0] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = old[i][j]
    return new
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

This is already optimal in time, O(m * n); its cost is the second matrix, O(m * n) extra space. A rectangle cannot be rotated in place (its shape changes), so it is the expected answer; the zip(reversed rows) one-liner is the same idea.

OPTIMAL SOLUTION · Rotate 90 degrees clockwise by reverse + transpose

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can a non-square matrix be rotated in place?

Not in a rectangular buffer, because the dimensions swap. You can do it in place only if the matrix is stored as a flat array and you permute the elements along cycles of the index map (i, j) to (j, m - 1 - i), which is O(mn) time but needs cycle bookkeeping. In an interview, allocate the new matrix and say why.

FOLLOW-UP 2 How would you rotate by k * 90 degrees?

Reduce k modulo 4 first. Then apply the single-turn function that many times, or write the four direct index maps so you only build the output once.

FOLLOW-UP 3 Rotate by k quarter turns.

Reduce k modulo 4 first; four turns are the identity. Apply the single-turn function that many times (at most three), or write the three index formulas directly to build the answer once.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# BASE QUESTION (optimal): Rotate 90 degrees clockwise by
# reverse + transpose
def rotate_cw(matrix):
    # reverse the rows, then swap rows and columns
    return [list(row) for row in zip(*matrix[::-1])]


# COUNTER-CLOCKWISE
def rotate_ccw(matrix):
    # swap rows and columns, then reverse the rows
    return [list(row) for row in zip(*matrix)][::-1]


# FOLLOW-UP 2, 3: How would you rotate by k * 90 degrees?
def rotate_k(matrix, k):
    # four quarter turns are the identity: use k mod 4
    for _ in range(k % 4):
        matrix = rotate_cw(matrix)
    return matrix


# INDEX RULE version
def rotate_by_index(matrix):
    # the same result from the index rule, built once:
    # new[j][m - 1 - i] = old[i][j]
    m, n = len(matrix), len(matrix[0]) if matrix else 0
    new = [[None] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = matrix[i][j]
    return new


if __name__ == "__main__":
    a = [[1, 2, 3], [4, 5, 6]]
    # the 2 x 3 example becomes 3 x 2
    assert rotate_cw(a) == [[4, 1], [5, 2], [6, 3]]
    assert rotate_by_index(a) == rotate_cw(a)
    assert rotate_ccw(a) == [[3, 6], [2, 5], [1, 4]]
    # a clockwise then an anticlockwise turn cancel out
    assert rotate_ccw(rotate_cw(a)) == a
    assert rotate_k(a, 4) == a
    assert rotate_k(a, 2) == [[6, 5, 4], [3, 2, 1]]
    assert rotate_k(a, 5) == rotate_cw(a)
    assert rotate_cw([[7]]) == [[7]]
```

```
    assert rotate_cw([]) == []
```

NOTES FROM CANDIDATES

The in-place rectangle rotation is essentially cycle-decomposition on the index permutation — explain it but don't burn time implementing.
Reported follow-up: rotate inside a window of a larger matrix while keeping the rest untouched.

PREPARATION

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Dynamic Programming (listed there as SKIP, no pages). Site tags: array, dp, medium

MEDIUM | 1Point3Acres | onsite-coding | 17 of 19

Jump Game with Profit

STUDY: TYPE | ALL TIME 56 | 30 DAYS 0 | PRIORITY 58/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #55 Jump Game

LeetCode match: LC 55 Jump Game (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/jump-game-with-profit

QUESTION

EXAMPLES

Variant-dependent; the principal recurrence is the one above.

APPROACH

Drill the reachability-jump, min-jumps, and coin-change family together so the DP shape is automatic.
Practice clarifying "can I skip without consuming a turn" before coding.

INTERVIEW-DAY PLAN

CLARIFY FIRST

This is underspecified, so ask: is arr[i] the maximum jump (a cap) or an exact jump, is profit collected on landing or when leaving, can you skip a cell for free, are revisits allowed, and what is the goal (reach the end, or maximize profit)?

BRUTE FORCE TO SAY OUT LOUD

Recursively try every jump from every cell. Exponential.

OPTIMAL IDEA

Pick a reading and say it. Cap reading: best[i] is the best profit standing on i; best[i] = profit[i] + max best[j] over j that can reach i; O(n * max jump), or O(n log n) with a heap and lazy deletion. Exact jump with optional skipping: dp[i] = max(dp[i + 1], profit[i] + dp[i + arr[i]]) from the right, O(n). Plain reachability (no profit) is a greedy farthest-reach scan in O(n).

DRY-RUN THESE CASES

A cell with jump 0, an unreachable end, negative profits, a single cell.

LIKELY FOLLOW-UPS

Minimum jumps (LeetCode 45), profits that can be negative, or a limit on the number of jumps.

PITFALLS

INTERVIEW-DAY PLAN

Treating the jump as a cap when it is exact, or the reverse. Not defining what skipping costs.

HOW TO THINK ABOUT IT

This statement is underspecified, so pin it down first (cap or exact jump, when profit is collected, can you skip, what is the goal). The reading with a maximum jump and profit on landing is a DP over positions: the best profit at i is its own profit plus the best of all positions that can reach it. The reading with an exact jump and a free skip is a one-line DP from the right.

Basis: Variant of LeetCode 55 (Jump Game) and LeetCode 45 (Jump Game II). The prompt is underspecified, so two readings are given; confirm which one with the interviewer.

BRUTE FORCE · Reading A: recursion over every jump length

BRUTE FORCE · PSEUDO-CODE

```
best = -INF
def go(i, total):
    global best
    if i == n - 1:
        best = max(best, total)
        return
    for step in range(1, arr[i] + 1):
        if i + step < n:
            gain = profit[i + step]
            go(i + step, total + gain)
```

TIME	SPACE
O(max jump ^ n)	O(n)

WHY IT IS SLOW

Every cell branches into up to max-jump choices, so the paths multiply (max jump to the power n). A table of the best profit per cell, filled left to right, is O(n * max jump).

BRUTE FORCE · Reading B: recursion over 'jump or skip' choices

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= n:
        return 0
    skip = best(i + 1)
    jump = 0
    if arr[i] > 0:
        jump = profit[i] + best(i + arr[i])
    return max(skip, jump)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

BRUTE FORCE · Reading B: recursion over 'jump or skip' choices

Two branches at every cell and the same suffix is solved repeatedly: O(2^n). Computing best(i) once per index from the right is O(n).

OPTIMAL SOLUTION · Reading A: arr[i] is a maximum jump, profit[i] is gained on landing, maximize total profit to the last index

TIME	SPACE
O(n * maxJump)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Reading B: arr[i] is an exact jump; at each index you may jump (gaining profit[i]) or skip one cell for free

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Speed up reading A to near-linear time.

The inner loop asks for the max of best[j] over all j that can still reach i. Maintain a max-heap of (best[j], j + arr[j]) and pop entries whose reach is below i (lazy deletion): O(n log n). If the jump limit is the same for all cells, a monotonic deque over a sliding window makes it O(n).

FOLLOW-UP 2 What if you want to know whether the end is reachable at all (plain Jump Game)?

Greedy: scan left to right keeping the farthest reachable index; if i exceeds it you are stuck. O(n) time, O(1) space. Use the DP when profits matter, and the greedy when only reachability matters.

FOLLOW-UP 3 Only reachability, no profit (LeetCode 55).

Greedy: keep the farthest index reachable so far; if the current index is beyond it you are stuck. O(n), O(1).

FOLLOW-UP 4 Make the profit DP faster.

best[i] needs the maximum of best[j] over j that can reach i. A max-heap with lazy deletion of positions that can no longer reach i makes it O(n log n); with a constant maximum jump a monotonic deque gives O(n).

FOLLOW-UP 5 Fewest jumps (LeetCode 45).

BFS by levels or a greedy that tracks the end of the current jump range and the farthest reach, O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq

NEG = float("-inf")


# SLOWER VERSION (say it first)
def max_profit_slow(arr, profit):
    # best[i]: most profit when landing on i (arr[j] is
    # the
    # farthest jump from j); O(n^2)
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
    for i in range(1, n):
        for j in range(i):
            # can j reach i? then landing on i is possible
            if best[j] != NEG and j + arr[j] >= i:
                total = best[j] + profit[i]
                best[i] = max(best[i], total)
    return -1 if best[-1] == NEG else best[-1]


# BASE QUESTION (optimal): Reading A: arr[i] is a maximum
# jump, profit[i] is gained on ...
def max_profit_fast(arr, profit):
    # a max-heap of (-best, reach) with lazy deletion
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
    heap = [(-best[0], arr[0])]
    for i in range(1, n):
        # forget cells that can no longer reach i
        while heap and heap[0][1] < i:
            heapq.heappop(heap)
        if heap:
            best[i] = -heap[0][0] + profit[i]
            heapq.heappush(heap, (-best[i], i + arr[i]))
    return -1 if best[-1] == NEG else best[-1]


# FOLLOW-UP 3: Only reachability, no profit (LeetCode 55).
# VARIANT: reachability only (LeetCode 55)
def can_reach_end(arr):
    # LeetCode 55: only reachability, greedy O(n)
    farthest = 0
```

```
    for i, jump in enumerate(arr):
        if i > farthest:
            return False
        farthest = max(farthest, i + jump)
    return True


# FOLLOW-UP 5: Fewest jumps (LeetCode 45).
# VARIANT: fewest jumps (LeetCode 45)
def fewest_jumps(arr):
    # LeetCode 45: levels of a BFS, greedy O(n)
    jumps = end = farthest = 0
    for i in range(len(arr) - 1):
        farthest = max(farthest, i + arr[i])
```

```
        if i == end:
            # we must jump now: extend to the farthest
            # reach
            jumps += 1
            end = farthest
    return jumps


if __name__ == "__main__":
    arr, profit = [2, 3, 1, 1, 4], [1, 5, 2, 3, 4]
    # best path: 0 -> 1 -> 2 -> 3 -> 4 = 15
    assert max_profit_slow(arr, profit) == 15
    assert max_profit_fast(arr, profit) == 15
    # the end cannot be reached
    assert max_profit_slow([1, 0, 1], [1, 1, 1]) == -1
    assert max_profit_fast([1, 0, 1], [1, 1, 1]) == -1
    assert can_reach_end(arr)
    assert not can_reach_end([1, 0, 1])
    assert fewest_jumps(arr) == 2
    # the fast version agrees with the slow one
    import itertools
    for jumps in itertools.product(range(4), repeat=5):
        pr = [1, 2, 3, 2, 1]
        assert max_profit_slow(jumps, pr) == (
            max_profit_fast(jumps, pr))
```

NOTES FROM CANDIDATES

Standard intern coding round, single source — clarification with interviewer is key (does "skip" advance by 1 or 0? are revisits allowed?).
Common bug: treating arr[i] as range cap rather than exact jump.

PREPARATION

Drill the reachability-jump, min-jumps, and coin-change family together so the DP shape is automatic.
Practice clarifying "can I skip without consuming a turn" before coding.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Dynamic Programming, Heaps, Stacks, Sorting and Binary Search (listed there as SKIP, no pages). Site tags: array, dp, binary-search, sorting, medium

MEDIUM | IN APP LC 354 | 1Point3Acres | phone-screen | 18 of 19

Russian Doll Envelopes

STUDY: TYPE | ALL TIME 52.8 | 30 DAYS 33 | PRIORITY 28/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: your app's interview approach (LeetCode #354)

LeetCode match: LC 354 Russian Doll Envelopes

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

QUESTION

LeetCode 354. Maximum chain of envelopes that strictly fit inside each other. Sort + LIS reduction.

Input: list of (width, height) envelopes.
Output: longest chain where each envelope strictly fits in the next.
Trick: sort by width ascending, height descending (when widths tie); run LIS (Longest Increasing Subsequence) on heights.
LIS with patience-sort gives O(n log n).

EXAMPLES

```
[[5,4],[6,4],[6,7],[2,3]] -> 3.
```

APPROACH

Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

HOW TO THINK ABOUT IT

Sort by width ascending so that only the heights matter, then find the longest strictly increasing subsequence of heights. Envelopes with equal width cannot nest, so sort those by height descending: then a strictly increasing run of heights can never use two envelopes of the same width.

Basis: LeetCode 354, Russian Doll Envelopes.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
envelopes.sort()  # by width
best = [1] * len(envelopes)
for i in range(len(envelopes)):
    for j in range(i):
        w1, h1 = envelopes[j]
        w2, h2 = envelopes[i]
        if w1 < w2 and h1 < h2:
            best[i] = max(best[i], best[j] + 1)
answer = max(best)
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each envelope is compared with all earlier ones, O(n^2). After sorting, a longest increasing subsequence of the heights with binary search is O(n log n).

OPTIMAL SOLUTION · Sort, then LIS with binary search

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why sort heights descending when widths tie?

If two envelopes have the same width they cannot nest, but an ascending height order would let the LIS pick both. Descending order makes a strictly increasing height run use at most one envelope per width.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from bisect import bisect_left


# BASE QUESTION (optimal): Sort, then LIS with binary
# search
def max_envelopes(envelopes):
    # sort by width; for equal widths put TALLER first so
    # a
    # strictly increasing height run can use only one of
    # them
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    # tails[k]: smallest ending height of a chain of k + 1
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)


# CHECKER: the O(n^2) version
def max_envelopes_slow(envelopes):
    # O(n^2) check used for testing
    ordered = sorted(envelopes)
    best = [1] * len(ordered)
    for i in range(len(ordered)):
        for j in range(i):
            wj, hj = ordered[j]
            wi, hi = ordered[i]
            if wj < wi and hj < hi:
```

```
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)


if __name__ == "__main__":
    envs = [[5, 4], [6, 4], [6, 7], [2, 3]]
    assert max_envelopes(envs) == 3
    assert max_envelopes([]) == 0
    # equal widths cannot nest, however the heights go
    assert max_envelopes([[1, 1], [1, 2], [1, 3]]) == 1
    assert max_envelopes([[4, 5], [4, 6], [6, 7], [2, 3],
                          [1, 1]]) == 4
```

```
    import itertools
    pts = [(w, h) for w in range(1, 4)
           for h in range(1, 4)]
    for combo in itertools.combinations(pts, 4):
        assert max_envelopes(list(combo)) == (
            max_envelopes_slow(list(combo)))
```

NOTES FROM CANDIDATES

Sorting heights descending on width-ties prevents same-width envelopes from chaining — easy to miss.
LIS via bisect_left on a tails array is the standard O(n log n) implementation.

PREPARATION

Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

STUDY HERE: topic 1, Arrays, Hashing and Two Pointers

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: hashmap, simulation, easy

EASY | 1Point3Acres | onsite-coding | 19 of 19

Robot Status Message Deduplication Window

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 33** | **PRIORITY 34/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (related)

Frequency: Low | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

QUESTION

Process timestamped robot messages and display a message only when it has not been displayed within the prior 10 seconds. Hidden arrivals do not refresh the last-displayed timestamp, and the exact 10-second boundary must be clarified.

Process a stream of (time, message) status events from a robot.
If the same message was displayed within the past 10 seconds, hide the new event.
Otherwise display it and update that message's last-displayed time.
Track the last time each message was displayed, not the last time it was received. A hidden duplicate must not refresh the window.
Clarify whether an event exactly 10 seconds after the prior display is allowed: the comparison may be > 10 or >= 10.

Logger API variant

Implement shouldPrintMessage(timestamp, message), returning true when the message has not been printed in the past 10 seconds and false otherwise.
Handle timestamps arriving out of order, including timestamp 12 arriving before timestamp 10.

EXAMPLES

```
time=10, message="solar panel activated" -> show
time=13, message="solar panel activated" -> hide
time=21, message="solar panel activated" -> show
```

The event at 21 compares against the display at 10, not the hidden arrival at 13.

APPROACH

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

HOW TO THINK ABOUT IT

The state that matters is the last time each message was displayed, not the last time it arrived. A message that is hidden must not move that time, otherwise a message repeated every few seconds would be hidden forever. A hash map from message to last displayed time answers each event in O(1).

Basis: LeetCode 359, Logger Rate Limiter (a message may be shown again only after the window has passed).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
shown = []  # (time, message) displayed
def on_event(t, msg):
    for t0, m0 in shown:
        if m0 == msg and t - t0 < 10:
            return False  # hidden
    shown.append((t, msg))
    return True  # displayed
```

TIME	SPACE
O(displayed) per event	O(displayed)

WHY IT IS SLOW

Each event scans the whole history. A map from message to its last displayed time answers each event in O(1).

OPTIMAL SOLUTION · Hash map of last displayed time

TIME	SPACE
O(1) per event	O(distinct messages)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Events can arrive out of order.

Decide the contract first: either a late event is judged against the stored time with abs(t - last), or you keep a sorted history per message and compare with the nearest displayed neighbours on both sides. The simple map is only correct for a chronological stream.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from bisect import bisect_left, bisect_right, insort


# BASE QUESTION (optimal): Hash map of last displayed time
class Dedup:
    """Chronological stream: remember the last display."""

    def __init__(self, window=10, boundary_allowed=True):
        self.window, self.last = window, {}
        # True: exactly `window` seconds later may show
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        # is this gap still inside the hiding window?
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        # only a displayed event starts a new window
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True


# FOLLOW-UP 1: Events can arrive out of order.
class DedupOutOfOrder:
    """Events may arrive late: look at both neighbours."""

    def __init__(self, window=10):
        self.window = window
        # message -> sorted list of displayed times
        self.shown = {}

    def show(self, t, msg):
        times = self.shown.setdefault(msg, [])
        # any display within `window` on either side?
        lo = bisect_right(times, t - self.window)
        hi = bisect_left(times, t + self.window)
        if lo < hi:
            return False
        insort(times, t)
        return True
```

```
if __name__ == "__main__":
    d = Dedup()
    # 13 is hidden and does not move the window
    assert [d.show(t, "solar") for t in (10, 13, 21)] == [
        True, False, True]
    # exactly 10 seconds later shows by default
    d = Dedup()
    d.show(0, "a")
    assert d.show(10, "a")
    # ... but is hidden if the boundary is not allowed
    d = Dedup(boundary_allowed=False)
    d.show(0, "a")
    assert not d.show(10, "a")
    o = DedupOutOfOrder()
    assert o.show(20, "x")
    # a late event close to a displayed one is hidden
    assert not o.show(15, "x")
    # a late event far enough from every display shows
    assert o.show(5, "x")
```

NOTES FROM CANDIDATES

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.
Resolve the exact-boundary rule before coding and make the comparison match that contract.

For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

PREPARATION

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.